Dendrite Evolution in Pure Melt – K.Bhagat

Theory, finite element formulation and the deal.II implementation
with automatic differentiation (`implicit_puremelt`)

27 October 2026

What this document covers. The solidification of a pure undercooled melt modelled with the phase-field method of Bhagat et al. (2023), Section 2.2. It goes in one line from physics to code: the sharp-interface Stefan problem → the free energy functionals → the phase-field strong form → the weak form → the finite element discretisation and Newton–Raphson solver → the Jacobian by automatic differentiation → the code architecture → a file-by-file documentation of implicit_puremelt → results. Every equation used by the code is derived here and every line of the residual is mapped back to its term.

Reference paper	K. Bhagat, S. Rudraraju, *Engineering with Computers* 39 (2023) 2345–2363
Code repository	`https://github.com/rathod-gaurav/Metal-Additive-Manufacturing/tree/main/Dendrite-KBhagat/implicit_puremelt`
Library	deal.II with Trilinos Sacado (automatic differentiation)

Contents

1 Overview **4**

2 The physical problem: the Stefan problem **4**
- 2.1 Sharp-interface equations . . . 4
- 2.2 The interface condition (1c): Gibbs–Thomson and kinetic undercooling . . . 5
- 2.3 Non-dimensional form . . . 5

3 Thermodynamic building blocks **5**
- 3.1 Gibbs free energy . . . 5
- 3.2 The phase field and the three building-block functions . . . 5
- 3.3 The enthalpy $U = u - \frac{1}{2}h(\phi)$ . . . 6
- 3.4 Why the temperature equation is written for U . . . 7

4 Free energy functionals **7**
- 4.1 The isothermal functional Π (Bhagat Eq. 3) . . . 7
- 4.2 The Lyapunov functional Π_L . . . 8
- 4.3 Governing equations from the functionals (Bhagat Eq. 4) . . . 8
- 4.4 Why two different functionals? . . . 9

5 The phase-field strong form **10**

5.1 Governing equations . . . 10
5.2 Anisotropy: $\lambda(\mathbf{n})$ and $\tau(\mathbf{n})$ . . . 10
5.3 Thin-interface limit and the parameter choice . . . 10

6 Weak form **11**
6.1 Function spaces . . . 11
6.2 Weak equations (Bhagat Eq. 6) . . . 11

7 Finite element discretisation and Newton's method **12**
7.1 Two DoFs per node . . . 12
7.2 Matrix–vector form of the residual . . . 12
7.3 The residual of node A and its six terms . . . 12
7.4 Newton–Raphson . . . 13

8 Automatic differentiation **13**
8.1 The idea . . . 13
8.2 The deal.II tools . . . 14
8.3 The five steps on one element . . . 14
8.4 What AD produces from each term . . . 14

9 Code architecture and workflow **15**
9.1 Files . . . 15
9.2 Workflow diagram . . . 15
9.3 What travels between the levels in one Newton iteration . . . 15

10 Code documentation **17**
10.1 CMakeLists.txt . . . 17
10.2 main.cpp — parameters and objects . . . 17
10.3 PureMelt.hpp and PureMelt.tpp — the problem class . . . 18
10.4 MakeGrid.tpp . . . 20
10.5 InitialConditions.hpp . . . 20
10.6 SetupSystem.tpp . . . 21
10.7 Solver.tpp — time loop and Newton loop . . . 22
10.8 AssembleSystem.tpp — Level 1 . . . 23
10.9 ComputeElement.tpp — Level 2: AD on one element . . . 24
10.10 ADConfig.hpp . . . 25
10.11 QPFields.hpp — the quadrature-point container . . . 25
10.12 EvaluateFields.tpp — Level 3a . . . 25
10.13 ElementResidual.tpp — Level 3b: the weak form . . . 26
10.14 ConstitutiveModel.hpp — the nonlinear laws . . . 27
10.15 OutputWriter.hpp/.tpp . . . 29

11 Results **29**
11.1 Dendrite evolution . . . 29
11.2 Animation . . . 29
11.3 Newton convergence . . . 29

12 Notes and possible extensions **31**

References **31**

A Full source listings **32**
CMakeLists.txt . . . 32
main.cpp . . . 32
PureMelt.hpp . . . 34

PureMelt.tpp . . . 35
MakeGrid.tpp . . . 36
InitialConditions.hpp . . . 36
SetupSystem.tpp . . . 37
Solver.tpp . . . 37
AssembleSystem.tpp . . . 38
ComputeElement.tpp . . . 39
ADConfig.hpp . . . 40
QPFields.hpp . . . 40
EvaluateFields.tpp . . . 40
ElementResidual.tpp . . . 41
ConstitutiveModel.hpp . . . 42
OutputWriter.hpp . . . 43
OutputWriter.tpp . . . 43

1 Overview

When a pure metal is cooled below its melting temperature T_m and a small solid seed is present, the seed does not grow as a sphere. Its surface becomes unstable and branches into a *dendrite*: a tree-like crystal whose main arms follow the preferred directions of the crystal lattice. Two physical effects decide its shape:

- **heat flow**: freezing releases latent heat, which must diffuse away from the interface before more liquid can solidify;
- **interface physics**: surface energy (Gibbs–Thomson effect), its dependence on crystal direction (anisotropy), and attachment kinetics.

Bhagat et al. solve this problem with a *phase-field* model: a smooth order parameter ϕ ($+1$ solid, -1 liquid) replaces the sharp interface, so the interface never has to be tracked. The model is that of Karma & Rappel (1998) as implemented by Plapp & Karma (2000). The code documented here solves it with

- two scalar fields per node, temperature u and phase field ϕ (deal.II `FESystem`);
- Q1 Lagrange elements, 2×2 Gauss quadrature;
- backward Euler in time and Newton–Raphson at every time step;
- the exact Newton Jacobian from automatic differentiation (Sacado), so only the residual is written by hand.

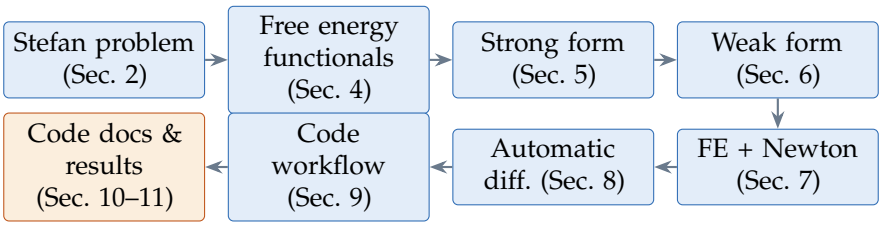


Figure 1: Road map of the document.

2 The physical problem: the Stefan problem

2.1 Sharp-interface equations

Let Ω_s and Ω_l be the solid and liquid regions, separated by the interface Γ with normal velocity v_n. Bhagat et al. (Eq. 1) write:

$$\frac{\partial T}{\partial t} = \frac{k}{c_p}\nabla^2 T, \qquad \mathbf{x} \in \Omega_s, \Omega_l, \tag{1a}$$

$$v_n L = k\big(\partial_n T|^{+} - \partial_n T|^{-}\big), \qquad \mathbf{x} \in \Gamma, \tag{1b}$$

$$T_m - T_i = \Gamma_g\,\kappa + \frac{v_n}{\mu_k}, \qquad \mathbf{x} \in \Gamma. \tag{1c}$$

(1a) is heat conduction in each phase; (1b), the *Stefan condition*, says that the latent heat released by the moving interface equals the jump in heat flux; (1c) fixes the interface temperature T_i.

2.2 The interface condition (1c): Gibbs–Thomson and kinetic undercooling

The interface sits slightly *below* T_m for two independent reasons; to first order their undercoolings add.

Curvature, $\Gamma_g\kappa$ (Gibbs–Thomson). An atom on a sharp convex tip has fewer solid neighbours than one on a flat surface, so it is held more weakly and needs a colder temperature to stay solid. Quantitatively, freezing a volume releases free energy $\Delta G_v = L\,\Delta T/T_m$ (Section 3.1), while growing a curved surface costs surface energy $\gamma\kappa$ per unit volume (for a circle of radius r grown by dr: $\gamma\, 2\pi\, dr/(2\pi r\, dr) = \gamma/r = \gamma\kappa$). Equilibrium gives

$$\frac{L\,\Delta T_\Gamma}{T_m} = \gamma\kappa \quad\Longrightarrow\quad \Delta T_\Gamma = \frac{\gamma T_m}{L}\,\kappa = \Gamma_g\kappa. \tag{1}$$

With $\Gamma_g = 2.36\times10^{-7}\,\mathrm{m\,K}$ and a $1\,\mu$m tip this is only $\approx 0.24\,\mathrm{K}$, yet it selects the tip radius. If γ depends on direction, $\gamma(\mathbf{n})$, so does Γ_g; this is what makes dendrite arms grow along preferred crystal directions.

Kinetics, v_n/μ_k. Atoms constantly join and leave the solid. At T_m the two rates balance; below it, joining wins. For small undercooling the net rate is linear in the undercooling, $v_n = \mu_k\,\Delta T_\mu$ (from linearising $v_n = v_0[1 - e^{-\Delta g/k_BT}]$). In metals μ_k is large and this term is small; the model below makes it vanish exactly by the choice of parameters (Section 5.3).

2.3 Non-dimensional form

With the scaled temperature $u = c_p(T - T_m)/L$, the paper's Eq. 2 reads

$$\frac{\partial u}{\partial t} = D\nabla^2 u, \qquad \mathbf{x}\in\Omega_s,\Omega_l, \tag{2a}$$

$$v_n = D\left(\partial_n u|^+ - \partial_n u|^-\right), \qquad \mathbf{x}\in\Gamma, \tag{2b}$$

$$u^* = -d(\mathbf{n})\,\kappa - \beta(\mathbf{n})\,v_n, \qquad \mathbf{x}\in\Gamma, \tag{2c}$$

with capillary length $d(\mathbf{n}) = \gamma(\mathbf{n})T_m c_p/L^2$ and kinetic coefficient $\beta(\mathbf{n})$. The driving force is the initial undercooling of the melt $\Delta = c_p(T_\infty - T_m)/L$; here $\Delta = -0.65$.

3 Thermodynamic building blocks

3.1 Gibbs free energy

The Gibbs free energy is the *definition* $G = H - TS$. Low enthalpy H (tight bonds) favours the solid; high entropy S (disorder) favours the liquid, more so when hot. At fixed temperature the phase with lower G wins. Comparing the phases, $\Delta G = G_l - G_s = \Delta H - T\Delta S$. At T_m they coexist, so $\Delta G = 0$ and $\Delta S = L/T_m$; below T_m, $\Delta G = L\,\Delta T/T_m > 0$ and the solid wins. The rule "lower G wins" holds only at *constant temperature*: this is the origin of the word *isothermal* in Section 4.

3.2 The phase field and the three building-block functions

The order parameter $\phi(\mathbf{x}, t)$ is $+1$ in the solid, -1 in the liquid, and varies smoothly across an interface of width $\sim\lambda$. Three functions of ϕ build the model. Because Bhagat et al. and Karma &

Rappel use the same letters for different things, this document uses the letters in the last column of Table 1.

Role	Bhagat et al.	Karma & Rappel	This document	In the code
double-well potential	$g(\phi)$	$f(\psi)$	g	inside `bulk_driving_force`
temperature tilt (coupling)	$h(\phi)$	$g(\psi)$	p	inside `bulk_driving_force`
latent-heat function	in $U = u - \frac{1}{2}\phi$	$h(\psi)$	h	`latent_heat_slope` (h')
bulk free energy density	$f(\phi, u)$	$f + \lambda u g$	f	derivative = `bulk_driving_force`
isothermal functional	Π	F_{iso}	Π	gives `element_residual`, ϕ rows
Lyapunov functional	Π_L	F	Π_L	gives `element_residual`, u rows

Table 1: Notation. The coupling constant ξ (Bhagat) is Karma & Rappel's λ; Bhagat's λ is the interface width W.

$g(\phi) = -\frac{1}{2}\phi^2 + \frac{1}{4}\phi^4$ **— double well.**
Two equal valleys at $\phi = \pm 1$: solid and liquid are the only stable states. Balanced against the gradient energy $\frac{1}{2}\lambda^2|\nabla\phi|^2$, it sets the interface width and the surface energy γ, hence the Gibbs–Thomson effect.

$p(\phi) = \phi - \frac{2}{3}\phi^3 + \frac{1}{5}\phi^5$, $p'(\phi) = (1-\phi^2)^2$ **— temperature tilt.**
Multiplied by ξu, it lowers one valley and raises the other (Fig. 2): $u < 0$ favours solid. It plays the role of the driving force $L\Delta T/T_m$. Because $p'(\pm 1) = 0$, the valleys stay exactly at ± 1 at every temperature.

$h(\phi) = \phi$ **— latent-heat function.**
States how much latent heat a point holds, through the enthalpy $U = u - \frac{1}{2}h(\phi)$ (Section 3.3). Bhagat et al. use the linear choice $h = \phi$, so $h' = 1$.

3.3 The enthalpy $U = u - \frac{1}{2}h(\phi)$

Step 1. Enthalpy per volume is sensible plus latent heat: $H_s = c_p T$, $H_l = c_p T + L$.

Step 2. With h running from -1 (liquid) to $+1$ (solid), the liquid fraction is $(1-h)/2$, so $H = c_p T + L\,(1 - h(\phi))/2$.

Step 3. With $c_p T = c_p T_m + Lu$, dividing by L and dropping constants:

$$\boxed{U = u - \tfrac{1}{2}h(\phi)} \qquad (\text{solid } U = u - \tfrac{1}{2},\ \text{liquid } U = u + \tfrac{1}{2}). \tag{2}$$

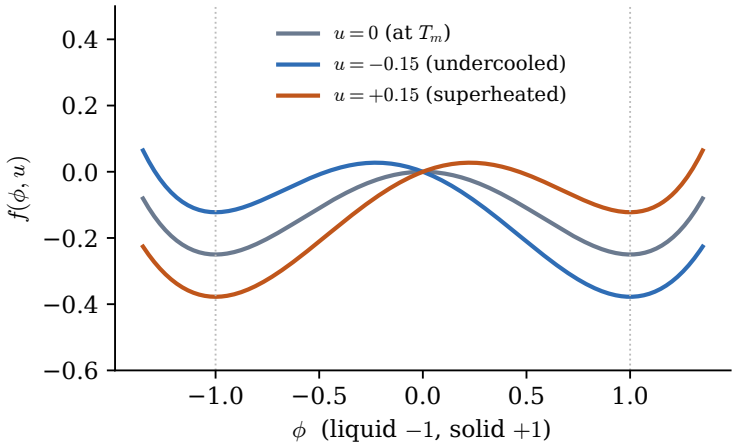


Figure 2: Bulk free energy $f(\phi, u) = g(\phi) + \xi u\, p(\phi)$, $\xi = 1.596$. Temperature tilts the double well but never moves its minima away from $\phi = \pm 1$.

The liquid holds one more unit of heat (L in scaled form); the $\frac{1}{2}$ appears because h spans two units. *Check:* freezing a point so fast that no heat leaves (fixed U) raises h by 2, hence u by 1, i.e. T by L/c_p: all latent heat warms the material.

3.4 Why the temperature equation is written for U

1. U **is conserved,** u **is not.** Enthalpy changes only by heat flowing across boundaries; temperature jumps wherever latent heat is released. A conservation law $\partial_t U = -\nabla \cdot \mathbf{J}$ is written for the conserved quantity.
2. **Latent heat is automatic.** Written in U there is no source term; it appears only when rewritten in u (Eq. 5), and exactly one unit of heat is released per unit of solid formed.
3. **It is the correct variational variable.** A conserved field evolves by a conserved gradient flow; and the physical variation of ϕ is one at *fixed* U (freezing cannot change the local enthalpy instantly, but it does change the temperature). This is what brings h' into the coupling (Section 4.4).

4 Free energy functionals

4.1 The isothermal functional Π (Bhagat Eq. 3)

A functional maps a whole field to one number. The total free energy of the domain is

$$\Pi[\phi, u] = \int_\Omega \left[f(\phi, u) + \tfrac{1}{2}\lambda^2(\mathbf{n})\, |\nabla\phi|^2 \right] dV, \qquad f(\phi, u) = -\tfrac{1}{2}\phi^2 + \tfrac{1}{4}\phi^4 + \xi u\, \phi\left(1 - \tfrac{2}{3}\phi^2 + \tfrac{1}{5}\phi^4\right). \quad (3)$$

It is called *isothermal* because u is treated as a fixed parameter: Π answers "at this temperature, which ϕ does the system prefer?". The interface term is nonzero only where ϕ varies; it is the surface energy through which the Gibbs–Thomson effect enters the model.

4.2 The Lyapunov functional Π_L

Temperature is not constant in solidification: latent heat warms the interface and diffuses. A *Lyapunov* functional is one that always decreases as the full coupled system evolves:

$$\Pi_L[\phi, U] = \int_\Omega \Big[g(\phi) + \xi u^2 + \tfrac{1}{2}\lambda^2(\mathbf{n})|\nabla\phi|^2 \Big] dV, \qquad u = U + \tfrac{1}{2}h(\phi). \tag{3}$$

h does not appear explicitly; it is hidden in u. The independent fields of Π_L are ϕ and U.

4.3 Governing equations from the functionals (Bhagat Eq. 4)

$$\frac{\partial U}{\partial t} = \nabla \cdot \Big(D \nabla \frac{\delta \Pi_L}{\delta u} \Big) \quad (4a), \qquad \tau(\mathbf{n}) \frac{\partial \phi}{\partial t} = -\frac{\delta \Pi}{\delta \phi} \quad (4b). \tag{4}$$

The non-conserved ϕ relaxes downhill; the conserved enthalpy diffuses down the gradient of its "chemical potential".

Derivation of Eq. 4a and of the heat equation (5a)

Energy conservation with Fourier's law, $\partial_t H = \nabla \cdot (k \nabla T)$, divided by L gives $\partial_t U = D\nabla^2 u$ ($D = k/c_p$). Since only ξu^2 in Π_L depends on U and $\partial u / \partial U|_\phi = 1$,

$$\frac{\delta \Pi_L}{\delta U}\Big|_\phi = 2\xi u \quad \Longrightarrow \quad \frac{\partial U}{\partial t} = \nabla \cdot \Big(\frac{D}{2\xi} \nabla \frac{\delta \Pi_L}{\delta U} \Big),$$

which is Eq. 4a up to the normalisation constant (Bhagat writes $\delta\Pi_L/\delta u$; at fixed ϕ it equals $\delta\Pi_L/\delta U$). It is pure energy conservation in variational notation, and with no boundary flux $\int (\delta\Pi_L/\delta U)\, \partial_t U \le 0$. Substituting $U = u - \frac{1}{2}h(\phi)$:

$$\frac{\partial u}{\partial t} = D\nabla^2 u + \tfrac{1}{2}h'(\phi)\frac{\partial \phi}{\partial t} \xrightarrow{h=\phi} \boxed{\frac{\partial u}{\partial t} = D\nabla^2 u + \frac{1}{2}\frac{\partial \phi}{\partial t}} \quad (5a) \tag{5}$$

Derivation of the phase-field equation (5b)

Write $\mathbf{q} = \nabla\phi$ and the gradient energy density $E(\mathbf{q}) = \frac{1}{2}\lambda^2(\mathbf{q})|\mathbf{q}|^2$ (λ depends on $\mathbf{q}$ through $\mathbf{n} = \mathbf{q}/|\mathbf{q}|$). The variational derivative of $\Pi = \int [f + E]$ is $\delta\Pi/\delta\phi = \partial f/\partial\phi - \nabla \cdot (\partial E/\partial \mathbf{q})$ with

$$\boldsymbol{\Theta} \equiv \frac{\partial E}{\partial \mathbf{q}} = \lambda^2 \nabla\phi + |\nabla\phi|^2 \lambda \frac{\partial \lambda}{\partial \nabla\phi}. \tag{6}$$

Hence

$$\boxed{\tau(\mathbf{n})\frac{\partial \phi}{\partial t} = -\frac{\partial f}{\partial \phi} + \nabla \cdot (\lambda^2 \nabla\phi) + \frac{\partial}{\partial x}\Big[|\nabla\phi|^2 \lambda \frac{\partial \lambda}{\partial(\partial_x \phi)} \Big] + \frac{\partial}{\partial y}\Big[|\nabla\phi|^2 \lambda \frac{\partial \lambda}{\partial(\partial_y \phi)} \Big]} \quad (5b) \tag{7}$$

with $\partial f/\partial\phi = g'(\phi) + \xi u\, p'(\phi) = -\phi + \phi^3 + \xi u (1-\phi^2)^2$.

4.4 Why two different functionals?

Approach 1 — one functional (Karma & Rappel's "variational form", VF). For consistency both equations would come from Π_L, with ϕ varied at fixed U. The term ξu^2 then depends on ϕ through $u = U + \frac{1}{2}h(\phi)$, and the chain rule gives the *coupling term*

$$\frac{\partial}{\partial \phi}\left[\xi u^2\right]_U = 2\xi u \cdot \tfrac{1}{2}h'(\phi) = \xi\, u\, h'(\phi). \tag{8}$$

The tilt and the latent heat are *locked together*: one function h does both jobs. With $h = \phi$ the coupling is the constant ξu, and $g'(\phi) + \xi u = -\phi + \phi^3 + \xi u \neq 0$ at $\phi = \pm 1$: the bulk values drift with temperature. Keeping them at ± 1 needs $h' \propto (1-\phi^2)^2$, i.e. $h_{VF} = \frac{15}{8}(\phi - \frac{2}{3}\phi^3 + \frac{1}{5}\phi^5)$. Approach 1 then works and guarantees

$$\frac{d\Pi_L}{dt} = -\int \tau\Big(\frac{\partial \phi}{\partial t}\Big)^2 - \int M\Big|\nabla \frac{\delta \Pi_L}{\delta U}\Big|^2 \leq 0.$$

Approach 2 — isothermal functional for ϕ ("isothermal variational formulation", IVF). ϕ comes from Π with coupling $\xi u\, p'(\phi) = \xi u(1-\phi^2)^2$, and the heat equation from energy conservation with $h = \phi$. Tilt and latent heat are chosen independently, and the minima stay at ± 1. The cost:

$$\frac{d\Pi}{dt} = -\int \tau\Big(\frac{\partial \phi}{\partial t}\Big)^2 + \underbrace{\int \xi\, p(\phi)\frac{\partial u}{\partial t}}_{\text{either sign}},$$

so no single functional is guaranteed to decrease. In Karma & Rappel's words, the model is "only variational isothermally in the trivial case where u is taken to be constant" — the case with no heat flow at all. This does not affect correctness: energy is conserved exactly and the thin-interface limit is the same.

	Approach 1 (VF)	Approach 2 (IVF) — **used by Bhagat and this code**
ϕ equation	$-\delta\Pi_L/\delta\phi\|_U$	$-\delta\Pi/\delta\phi$
coupling term	$\xi u\, h'(\phi)$, tied to h	$\xi u(1-\phi^2)^2$
latent heat source	$\frac{1}{2}h'_{VF}\,\partial_t\phi$	$\frac{1}{2}\partial_t\phi$
functional decreases	always	not guaranteed
grid convergence	slower; spurious steady branch at high V	faster ($\Delta x/W_0 = 0.4$ sufficed)
thin-interface limit	Stefan problem	same Stefan problem

Table 2: The two formulations (Karma & Rappel 1998, Sec. II and V).

Approach 2 is cheaper not per time step (cost is the same) but because results converge on a coarser grid: doubling Δx in 3D gives 8 times fewer nodes and, for explicit stepping, 4 times fewer steps. The three references the paper cites contribute as follows: Karma & Rappel [42] define both formulations and the thin-interface relations; Plapp & Karma [13] use the IVF as is; Boettinger et al. [41] describe the general recipe (entropy functional, or an isothermal free energy plus an appended heat equation).

5 The phase-field strong form

5.1 Governing equations

In $\Omega = \Omega_s \cup \Omega_l$ (the whole domain; no interface is tracked):

$$\frac{\partial u}{\partial t} = D\nabla^2 u + \frac{1}{2}\frac{\partial \phi}{\partial t}, \tag{5a}$$

$$\tau(\mathbf{n})\frac{\partial \phi}{\partial t} = -\frac{\partial f}{\partial \phi} + \nabla \cdot \boldsymbol{\Theta}(\nabla\phi), \qquad \boldsymbol{\Theta} = \lambda^2 \nabla\phi + |\nabla\phi|^2 \lambda \frac{\partial \lambda}{\partial \nabla\phi}, \tag{5b}$$

with zero-flux conditions $\nabla u \cdot \mathbf{m} = 0$, $\boldsymbol{\Theta} \cdot \mathbf{m} = 0$ on $\partial\Omega$ ($\mathbf{m}$ the outward normal), $u(\mathbf{x}, 0) = \Delta$, and a solid seed in $\phi(\mathbf{x}, 0)$.

5.2 Anisotropy: $\lambda(\mathbf{n})$ and $\tau(\mathbf{n})$

$$\lambda(\mathbf{n}) = \lambda_0\, a_s(\mathbf{n}), \qquad \tau(\mathbf{n}) = \tau_0\, a_s^2(\mathbf{n}), \qquad \mathbf{n} = \frac{\nabla\phi}{|\nabla\phi|},$$

$$a_s = (1 - 3\epsilon_4)\left[1 + \frac{4\epsilon_4}{1 - 3\epsilon_4}\,\frac{(\partial_x\phi)^4 + (\partial_y\phi)^4}{|\nabla\phi|^4}\right] = 1 + \epsilon_4 \cos(4\theta), \qquad \tan\theta = \partial_y\phi / \partial_x\phi. \tag{9}$$

(Equivalence: $n_x^4 + n_y^4 = (3 + \cos 4\theta)/4$.) The first form is used in the code because it needs only derivatives of ϕ. λ sets the interface width and the surface energy ($\gamma \propto \lambda$), so $\gamma(\mathbf{n}) = \gamma_0 a_s(\mathbf{n})$ with four-fold symmetry. Dendrite arms grow along the directions of *maximum* a_s (0°, 90°, …) because there the surface stiffness $a_s + a_s'' = 1 - 15\epsilon_4 \cos 4\theta$ is smallest (Fig. 3). The square in $\tau \propto a_s^2$ keeps the kinetic coefficient zero in every direction (next subsection).

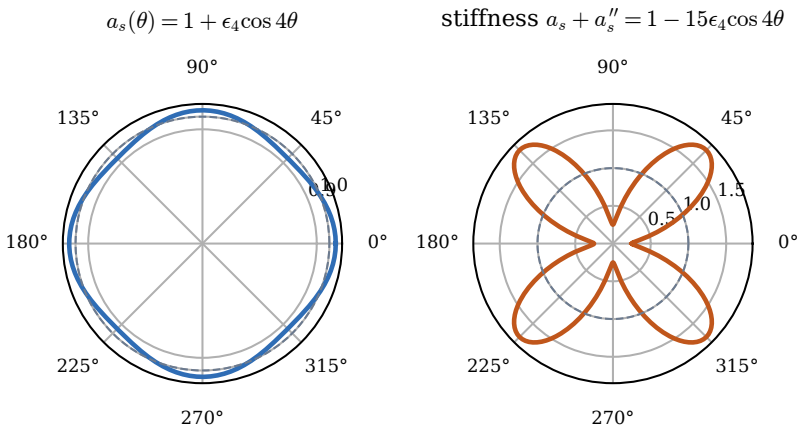


Figure 3: Anisotropy for $\epsilon_4 = 0.05$: surface energy factor a_s (left) and stiffness (right).

5.3 Thin-interface limit and the parameter choice

Karma & Rappel show that, for an interface of finite width $W = \lambda_0$, the model reproduces Eq. (2c) with

$$d_0 = a_1 \frac{W}{\xi}, \qquad \beta = a_1\left(\frac{\tau}{\xi W} - a_2 \frac{W}{D}\right), \qquad a_1 = 0.8839,\ a_2 = 0.6267. \tag{10}$$

Setting $\beta = 0$ (no kinetic undercooling, appropriate for metals) gives $\tau_0 = a_2\,\xi\,W^2/D$. With the paper's values $W = \lambda_0 = 1$, $\xi = 1.596$, $D = 1$: $\tau_0 = 0.6267 \times 1.596 = 1.0002 \approx 1$ ✓, and $d_0/\lambda_0 = a_1/\xi = 0.554$ (the paper's text quotes 0.544; Karma & Rappel's Fig. 5 uses 0.554). The parameters in main.cpp are therefore exactly the zero-kinetics set.

Parameter	Symbol	Value	In main.cpp
thermal diffusivity	D	1	`D`
anisotropy strength	ϵ_4	0.05	`eps4`
interface width	λ_0	1	`lambda0`
relaxation time	τ_0	1	`tau0`
coupling parameter	ξ	1.596	`xi`
undercooling	Δ	-0.65	`undercooling`

Table 3: Model parameters (Bhagat Table 1 and Sect. 4.1, after Karma & Rappel).

6 Weak form

6.1 Function spaces

Find $u \in \mathcal{S}_u$, $\phi \in \mathcal{S}_\phi$ with

$$\mathcal{S}_u = \{u \in H^1(\Omega) \,|\, u = u' \text{ on } \Gamma^u\}, \qquad \mathcal{S}_\phi = \{\phi \in H^1(\Omega) \,|\, \phi = \phi' \text{ on } \Gamma^\phi\},$$

such that the weak equations hold for all

$$w_u \in \mathcal{V}_u = \{w_u \in H^1(\Omega) \,|\, w_u = 0 \text{ on } \Gamma^u\}, \qquad w_\phi \in \mathcal{V}_\phi = \{w_\phi \in H^1(\Omega) \,|\, w_\phi = 0 \text{ on } \Gamma^\phi\}.$$

- $H^1(\Omega)$: functions that are square-integrable and have square-integrable first derivatives. Only first derivatives appear in the weak form, so H^1 is exactly what makes every integral finite; it allows kinks but not jumps, so C^0 Q1 elements are admissible.
- Γ^u, Γ^ϕ: parts of the *outer* boundary where values are prescribed (Dirichlet). They are not the solid–liquid interface Γ. Test functions vanish there because the solution is already known.
- In this problem all boundaries are zero-flux, so $\Gamma^u = \Gamma^\phi = \emptyset$ and all four spaces are $H^1(\Omega)$.

6.2 Weak equations (Bhagat Eq. 6)

Multiply (5a) by w_u and (5b) by w_ϕ, integrate over Ω, and integrate the divergence terms by parts. The boundary integrals vanish (zero flux):

$$\int_\Omega w_u \Big(\frac{\partial u}{\partial t} - \frac{1}{2}\frac{\partial \phi}{\partial t}\Big) dV + \int_\Omega D\,\nabla w_u \cdot \nabla u\, dV = 0, \tag{6a}$$

$$\int_\Omega w_\phi \Big(\tau \frac{\partial \phi}{\partial t} + \frac{\partial f}{\partial \phi}\Big) dV + \int_\Omega \lambda^2 \nabla w_\phi \cdot \nabla \phi\, dV + \int_\Omega |\nabla \phi|^2 \lambda \Big(\frac{\partial w_\phi}{\partial x}\frac{\partial \lambda}{\partial(\partial_x \phi)} + \frac{\partial w_\phi}{\partial y}\frac{\partial \lambda}{\partial(\partial_y \phi)}\Big) dV = 0. \tag{6b}$$

The last two integrals of (6b) together are $\int \nabla w_\phi \cdot \boldsymbol{\Theta}\, dV$; the code uses this combined form.

7 Finite element discretisation and Newton's method

7.1 Two DoFs per node

Both fields use the same Q1 basis N_B:

$$u_h = \sum_B N_B\, u_B, \qquad \phi_h = \sum_B N_B\, \phi_B, \qquad \mathbf{d} = \begin{bmatrix} \mathbf{u} \\ \phi \end{bmatrix}.$$

In deal.II this is `FESystem<Nsd> fe(FE_Q<Nsd>(1), 2)`: two copies of a scalar Q1 element, component 0 for u and component 1 for ϕ. Each Q1 cell therefore has 4 nodes $\times$ 2 fields $=$ 8 local DoFs, and a mesh of 250×250 cells has $2 \times 251^2 = 126{,}002$ DoFs.

7.2 Matrix–vector form of the residual

With backward Euler (t^{n+1} unknowns, t^n known) and $h' = 1$:

$$\mathbf{R}_u = \frac{1}{\Delta t}\mathbf{M}(\mathbf{u} - \mathbf{u}^n) - \frac{1}{2\Delta t}\mathbf{M}(\phi - \phi^n) + D\,\mathbf{K}\,\mathbf{u}, \tag{11}$$

$$\mathbf{R}_\phi = \frac{1}{\Delta t}\mathbf{M}^\tau(\phi)(\phi - \phi^n) + \mathbf{F}(\phi, \mathbf{u}) + \mathbf{Q}(\phi), \tag{12}$$

Symbol	Entry	Meaning
$\mathbf{M}$	$\int N_A N_B$	mass matrix
$\mathbf{K}$	$\int \nabla N_A \cdot \nabla N_B$	stiffness matrix
$\mathbf{M}^\tau(\phi)$	$\int \tau(\nabla\phi_h) N_A N_B$	τ-weighted mass matrix
$\mathbf{F}(\phi, \mathbf{u})$	$\int N_A\, \partial f/\partial\phi(\phi_h, u_h)$	bulk driving force
$\mathbf{Q}(\phi)$	$\int \nabla N_A \cdot \boldsymbol{\Theta}(\nabla\phi_h)$	gradient + anisotropy

7.3 The residual of node A and its six terms

Each node A has two residual entries:

$$R_{u,A} = \underbrace{\int N_A \frac{u_h - u_h^n}{\Delta t}}_{T_1 \text{ heat storage}} \underbrace{- \int N_A\, \tfrac{1}{2} h' \frac{\phi_h - \phi_h^n}{\Delta t}}_{T_2 \text{ latent heat}} + \underbrace{\int \nabla N_A \cdot D \nabla u_h}_{T_3 \text{ conduction}}, \tag{13}$$

$$R_{\phi,A} = \underbrace{\int N_A\, \tau \frac{\phi_h - \phi_h^n}{\Delta t}}_{T_4 \text{ relaxation}} + \underbrace{\int N_A \frac{\partial f}{\partial \phi}}_{T_5 \text{ bulk driving force}} + \underbrace{\int \nabla N_A \cdot \boldsymbol{\Theta}}_{T_6 \text{ interface energy}}. \tag{14}$$

Every term is $\int$(test-function part) $\times$ (field part). Grouping by test-function part, with quadrature $\int(\cdot) \approx \sum_q (\cdot)|_{\mathbf{x}_q} JxW_q$:

$$R_{u,A} = \sum_q \left[N_A\, s_u + \nabla N_A \cdot \mathbf{v}_u \right] JxW_q, \qquad R_{\phi,A} = \sum_q \left[N_A\, s_\phi + \nabla N_A \cdot \mathbf{v}_\phi \right] JxW_q, \tag{15}$$

$$s_u = \tfrac{u_h - u_h^n}{\Delta t} - \tfrac{1}{2} h' \tfrac{\phi_h - \phi_h^n}{\Delta t}, \quad \mathbf{v}_u = D\nabla u_h, \quad s_\phi = \tau \tfrac{\phi_h - \phi_h^n}{\Delta t} + \tfrac{\partial f}{\partial \phi}, \quad \mathbf{v}_\phi = \boldsymbol{\Theta}.$$

The structure of the code follows from this. $s_u, \mathbf{v}_u, s_\phi, \mathbf{v}_\phi$ depend on the quadrature point but not on A: compute them once per point (outer loop). $N_A, \nabla N_A$ depend on A: loop over them inside (inner loop).

7.4 Newton–Raphson

At each time step solve $\mathbf{R}(\mathbf{d}^{n+1}) = \mathbf{0}$ by repeating

$$\mathbf{J}\,\Delta\mathbf{d} = -\mathbf{R}, \qquad \mathbf{d} \leftarrow \mathbf{d} + \Delta\mathbf{d}, \qquad \mathbf{J} = \frac{\partial \mathbf{R}}{\partial \mathbf{d}} = \begin{bmatrix} \mathbf{J}_{uu} & \mathbf{J}_{u\phi} \\ \mathbf{J}_{\phi u} & \mathbf{J}_{\phi\phi} \end{bmatrix},$$

with everything evaluated at the current iterate $\mathbf{d}^{n+1,(k)}$ (`sol_k`); the old values $\mathbf{d}^n$ (`sol_n`) are fixed data and enter only the time differences. The Jacobian blocks are

$$\begin{aligned} \mathbf{J}_{uu} &= \tfrac{1}{\Delta t}\mathbf{M} + D\mathbf{K}, \qquad \mathbf{J}_{u\phi} = -\tfrac{1}{2\Delta t}\mathbf{M}, \\ (\mathbf{J}_{\phi u})_{AB} &= C_{AB} = \int \xi (1-\phi_h^2)^2 N_A N_B, \\ (\mathbf{J}_{\phi\phi})_{AB} &= \tfrac{1}{\Delta t} M^{\tau}_{AB} + \tfrac{1}{\Delta t}\int N_A(\phi_h - \phi_h^n)\frac{\partial \tau}{\partial \mathbf{q}} \cdot \nabla N_B + G_{AB} + \int \nabla N_A \cdot \mathbf{H}\,\nabla N_B, \end{aligned}$$

where $G_{AB} = \int [-1 + 3\phi_h^2 - 4\xi u_h \phi_h (1-\phi_h^2)] N_A N_B$ is the mass matrix weighted by $\partial^2 f/\partial\phi^2$, and $\mathbf{H} = \partial^2 E/\partial\mathbf{q}\partial\mathbf{q}$:

$$H_{ij} = \lambda^2 \delta_{ij} + 2\lambda\Big(q_i \frac{\partial \lambda}{\partial q_j} + \frac{\partial \lambda}{\partial q_i} q_j\Big) + |\mathbf{q}|^2 \Big(\frac{\partial \lambda}{\partial q_i}\frac{\partial \lambda}{\partial q_j} + \lambda \frac{\partial^2 \lambda}{\partial q_i \partial q_j}\Big).$$

Why G is a matrix. $F_A = \int N_A\, s(\phi_h, u_h)$ with $s = \partial f/\partial\phi$. Differentiating under the integral and using $\partial\phi_h/\partial\phi_B = N_B$: $\partial F_A/\partial\phi_B = \int N_A\,(\partial s/\partial\phi)\,N_B = G_{AB}$ — one number per pair (A, B), the Jacobian of the vector $\mathbf{F}$. $\mathbf{C}$ follows identically with $\partial u_h/\partial u_B = N_B$ and $\partial s/\partial u = \xi(1-\phi^2)^2$.

Anisotropy derivatives. With $S = (q_x^4 + q_y^4)/|\mathbf{q}|^4$, $a_s = (1 - 3\epsilon_4) + 4\epsilon_4 S$:

$$\frac{\partial S}{\partial q_x} = \frac{4 q_x q_y^2 (q_x^2 - q_y^2)}{|\mathbf{q}|^6}, \qquad \frac{\partial S}{\partial q_y} = \frac{4 q_x^2 q_y (q_y^2 - q_x^2)}{|\mathbf{q}|^6},$$

and $\mathbf{q}\cdot\partial\lambda/\partial\mathbf{q} = 0$ because λ depends only on direction. Writing $\mathbf{J}_{\phi\phi}$ by hand requires the second derivatives of S too; automatic differentiation removes that work.

8 Automatic differentiation

8.1 The idea

An AD number carries a value *and* its derivatives; each arithmetic operation applies the chain rule to both. For $f(x) = x^2$ at $x = 3$, start from $(3, 1)$; $x \cdot x$ gives $(9, 1\cdot 3 + 3\cdot 1) = (9, 6)$, i.e. $f(3) = 9$, $f'(3) = 6$, exactly. On a Q1 cell each AD number carries 8 derivative slots, one per local unknown. If the residual is computed from such numbers, each residual entry carries its row of the cell Jacobian.

Seeding. Unknown j is given a 1 in slot j and 0 elsewhere ($\partial d_j/\partial d_k = \delta_{jk}$). Interpolation then gives $\partial\phi_h(\mathbf{x}_q)/\partial d_j = N_j(\mathbf{x}_q)$ automatically — the step used by hand to derive $\mathbf{G}$.

8.2 The deal.II tools

Name	Meaning
`sacado_dfad`	Trilinos Sacado, forward mode, number of derivatives set at run time
`ResidualLinearization<ADTypeCode,double>`	helper: give it $\mathbf{R}(\mathbf{d})$, it returns $\mathbf{R}$ and $\partial\mathbf{R}/\partial\mathbf{d}$
`ADNumber`	the number type that carries value + derivatives

`ResidualLinearization` (not `EnergyFunctional`) is the right helper because the model is not derived from one functional (Approach 2).

8.3 The five steps on one element

1. **Fresh helper**: `ADHelper ad_helper(8, 8)` — 8 unknowns in, 8 residual entries out.
2. **Seed**: `register_dof_values(sol_k, local_dof_indices)` copies the element's current iterate into 8 AD numbers, each tagged "I am unknown j".
3. **Fields at quadrature points**: $u_h, \phi_h, \nabla u_h, \nabla \phi_h$ from the AD values (tracked); u_h^n, ϕ_h^n from `sol_n` as `double` (not tracked).
4. **Residual**: Eqs. 6a–6b written with these numbers; every entry is an AD number.
5. **Read out**: `compute_residual` (values $\to \mathbf{R}$), `compute_linearization` (derivatives $\to \mathbf{J}$).

Why the old values are `double`. Newton differentiates with respect to $\mathbf{d}^{n+1}$. The old values are fixed during the step, so their derivatives are zero; as `double`s they add value but no derivative, exactly $\partial\phi^n/\partial\phi = 0$. Computing them from the seeded values would instead use the current iterate and make the time derivative vanish.

8.4 What AD produces from each term

Term	Jacobian block produced automatically
T_1	$\mathbf{M}/\Delta t$ in $\mathbf{J}_{uu}$
T_2	$-\mathbf{M}/(2\Delta t)$ in $\mathbf{J}_{u\phi}$
T_3	$D\mathbf{K}$ in $\mathbf{J}_{uu}$
T_4	$\mathbf{M}^\tau/\Delta t$ and the $\partial\tau/\partial\mathbf{q}$ term in $\mathbf{J}_{\phi\phi}$
T_5	$\mathbf{G}$ in $\mathbf{J}_{\phi\phi}$ and $\mathbf{C}$ in $\mathbf{J}_{\phi u}$
T_6	$\int \nabla N_A \cdot \mathbf{H}\, \nabla N_B$ in $\mathbf{J}_{\phi\phi}$

Rules for AD code. (1) Anything that depends on the unknowns must be `ADNumber`; a cast to `double` silently drops derivatives. (2) Constitutive functions must be templates on the number type, not `std::function<double(double)>`, which converts to `double`. (3) Sacado uses expression templates: store scalar products in a named `ADNumber` before multiplying a `Tensor` by them (`lambda2`, `coef`). (4) Regularise singular operations: a_s divides by $|\nabla\phi|^4$, so $\delta = 10^{-12}$ is added to $|\nabla\phi|^2$.

9 Code architecture and workflow

9.1 Files

The code follows a one-function-per-file layout: main.cpp sets parameters; the class header PureMelt.hpp declares the problem and includes the .tpp files that define its member functions; the AD layer is split by responsibility.

File	Defines	Role
main.cpp	`main()`	parameters; builds `ConstitutiveModel`, `OutputWriter`, `PureMelt`; calls `run()`
PureMelt.hpp/.tpp	class, constructor, `run()`	data members, extractors, call sequence
MakeGrid.tpp	`make_grid()`	250×250 square mesh on $[-250, 250]^2$
InitialConditions.hpp	`InitialConditions`	$u_0 = \Delta$, tanh seed for ϕ_0
SetupSystem.tpp	`setup_system()`	DoFs, sparsity, vectors, initial condition
Solver.tpp	`solve()`	time loop, Newton loop, linear solve, output
AssembleSystem.tpp	`assemble_system()`	Level 1: loop over elements, scatter-add
ComputeElement.tpp	`compute_element()`	Level 2: AD bookkeeping on one element
ADConfig.hpp	`AD::ADHelper`, `AD::ADNumber`	the single place the AD backend is chosen
QPFields.hpp	`QPFields<Nsd,Number>`	container: fields at quadrature points
EvaluateFields.tpp	`evaluate_fields<Number>()`	Level 3a: nodal values $\rightarrow$ fields at $\mathbf{x}_q$
ElementResidual.tpp	`element_residual<Number>()`	Level 3b: Eqs. 6a–6b (the physics)
ConstitutiveModel.hpp	`ConstitutiveModel`	nonlinear laws: $\partial f/\partial\phi$, h', a_s, λ, τ
OutputWriter.hpp/.tpp	`OutputWriter`	VTU files per step and a PVD collection

9.2 Workflow diagram

9.3 What travels between the levels in one Newton iteration

#	Object	Type	Made in	Used in
1	`sol_k`, `sol_n`	global `Vector<double>`	Solver.tpp	step 2 (`sol_k`), step 3 (`sol_n`)
2	`dof_values_ad`	8 seeded `ADNumber`s	step 2	step 3
3	`fields`	`QPFields<Nsd,ADNumber>`	step 3	step 4
4	`residual_ad`	8 `ADNumber`s (value + derivatives)	step 4	step 5
5	`Rlocal`, `Jlocal`	`Vector`, `FullMatrix<double>`	step 5	Level 1 scatter
6	`NR_residual`, `NR_jacobian`	global vector, sparse matrix	Level 1	convergence check, solve
7	`NR_update`	global `Vector<double>`	linear solve	update of `sol_k`

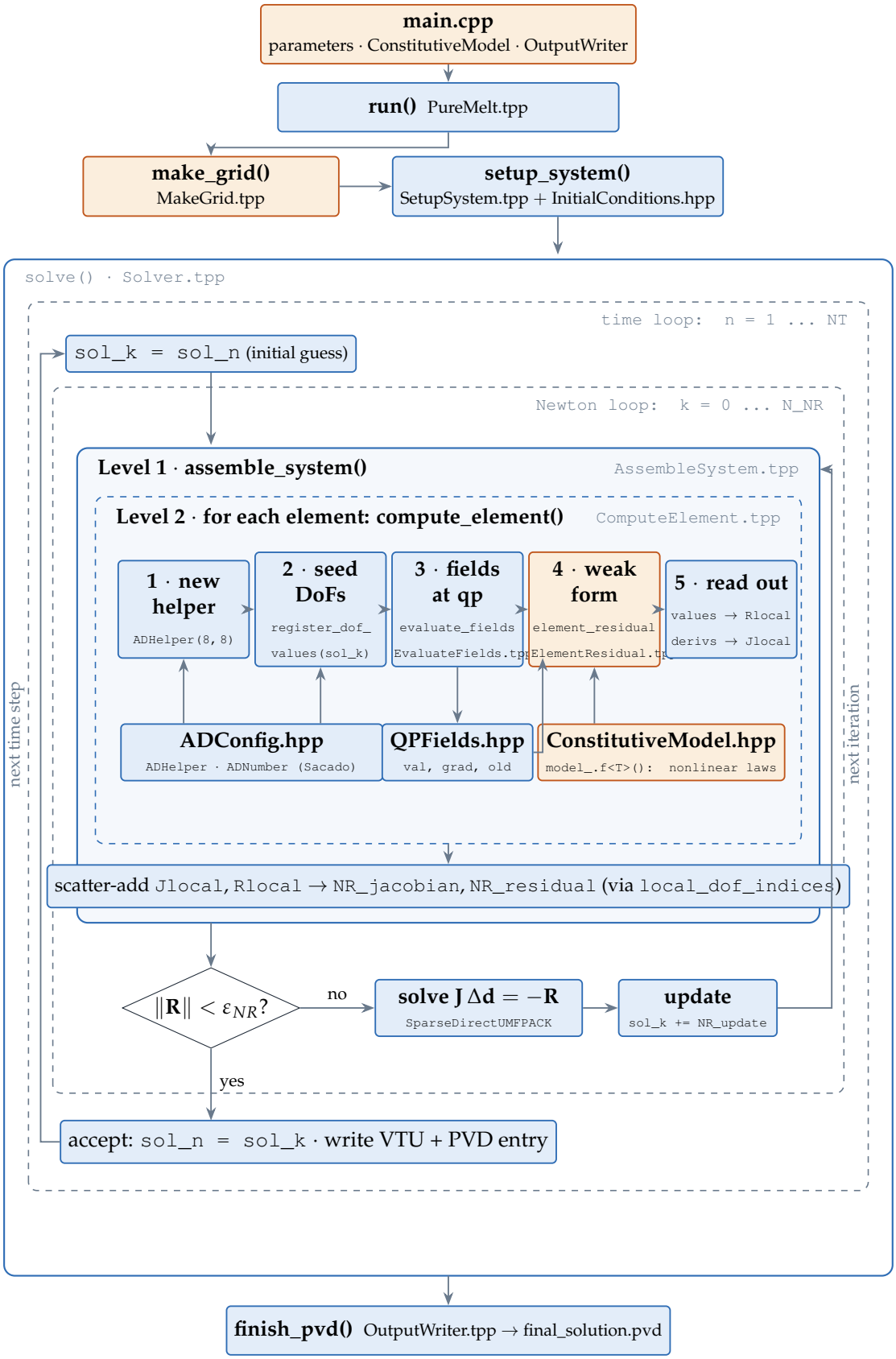


Figure 4: Workflow of implicit_puremelt. Blue: framework files reused as is for any problem. Orange: problem-specific files edited for new physics. Dashed boxes are loops.

10 Code documentation

Line numbers in the listings are those of the files in the repository. Full listings are in Appendix A.

10.1 CMakeLists.txt

Builds one executable `main` from main.cpp, adds include/ to the include path, and links deal.II (`DEAL_II_SETUP_TARGET`). It requests deal.II 9.7.1 at a local installation path (`HINTS`). deal.II must be configured with Trilinos and Sacado for ADConfig.hpp to compile.

10.2 main.cpp — parameters and objects

```
38 int main(){
39     constexpr unsigned int Nsd = 2;
40     constexpr unsigned int BfOrder = 1;
41     const unsigned int quadOrder = BfOrder + 1;
42 
43     //Mesh
44     const double x_ll = -250.0;
45     const double x_ul = 250.0;
46     const unsigned int n_elements = 250; //number of elements per side
47 
48     //NR parameters
49     const unsigned int NT = 100; //total number of timesteps
50     const double dt = 0.01; //timestep size
51     const unsigned int N_NR = 10; //total number of newton iterates per timestep
52     const double epsilon_NR = 1e-6; //NR convergence tolerance
53 
54     //Field variables of the problem (component index inside FESystem)
55     static constexpr unsigned int TEMP = 0; //u : scaled temperature
56     static constexpr unsigned int PHI = 1; //phi : phase field order parameter
57     static constexpr unsigned int N_FIELDS = 2; //total two fields in this problem
58 
59     //Initial conditions
60     const double undercooling = -0.65;
61     Point<Nsd> seed_center;
62     seed_center[0] = 0.0; seed_center[1] = 0.0; //seed_center[2] = 0.0;
63     const double seed_radius = 5.0;
64 
65     const double D = 1.0;
66     const double eps4 = 0.05;
67     const double lambda0 = 1.0;
68     const double tau0 = 1.0;
69     const double xi = 1.596;
70     ConstitutiveModel model(D, eps4, lambda0, tau0, xi);
71 
72     OutputWriter<Nsd,BfOrder> output_writer("output");
73 
74     PureMelt<Nsd,BfOrder> problem(
75         x_ll, x_ul, n_elements, quadOrder,
76         TEMP, PHI, N_FIELDS, undercooling, seed_center, seed_radius,
77         NT, dt, N_NR, epsilon_NR,
78         model, output_writer
79     );
80 
```

```
81      problem.run();
```

Variable	Value	Meaning
`Nsd`, `BfOrder`, `quadOrder`	2, 1, 2	2D, Q1 elements, 2×2 Gauss points
`x_ll`, `x_ul`, `n_elements`	-250, 250, 250	domain $[-250, 250]^2$, mesh size $h = 2$
`NT`, `dt`	100, 0.01	time steps and step size ($t_{\text{end}} = 1$)
`N_NR`, `epsilon_NR`	10, 10^{-6}	max. Newton iterations, absolute tolerance on $\|\mathbf{R}\|_2$
`TEMP`, `PHI`, `N_FIELDS`	0, 1, 2	FESystem component of each field
`undercooling`	-0.65	$u_0 = \Delta$
`seed_center`, `seed_radius`	(0,0), 5	initial solid seed
`D`, `eps4`, `lambda0`, `tau0`, `xi`	1, 0.05, 1, 1, 1.596	model parameters

The `ConstitutiveModel` and `OutputWriter` are created here and passed to `PureMelt` by reference, so the problem class never hard-codes material laws or output paths.

10.3 PureMelt.hpp and PureMelt.tpp — the problem class

```
8   template<unsigned int Nsd, unsigned int BfOrder>
9   class PureMelt{
10      public:
11          PureMelt(
12              const double x_ll, const double x_ul, const unsigned int n_elements,
    const unsigned int quadOrder, //mesh
13              const unsigned int TEMP, const unsigned int PHI, const unsigned int
    N_FIELDS,
14              const double undercooling, Point<Nsd> seed_center, const double
    seed_radius, //initial conditions
15              const unsigned int NT, const double dt, const unsigned int N_NR, const
     double epsilon_NR, //newton solver
16              const ConstitutiveModel& model, OutputWriter<Nsd,BfOrder>&
    output_writer
17          );
18          void run();
19
20      private:
21          const double x_ll_, x_ul_;
22          const unsigned int n_elements_;
23          const unsigned int quadOrder_;
24          const unsigned int TEMP_;
25          const unsigned int PHI_;
26          const unsigned int N_FIELDS_;
27          const double undercooling_;
28          Point<Nsd> seed_center_;
29          const double seed_radius_;
30          const unsigned int NT_;
31          const double dt_;
32          const unsigned int N_NR_;
33          const double epsilon_NR_;
34          const ConstitutiveModel& model_;
35          OutputWriter<Nsd,BfOrder>& output_writer_;
36
37          void make_grid();
38          void setup_system();
```

```
        void assemble_system();
        void compute_element(
            const typename DoFHandler<Nsd>::active_cell_iterator& elem, FEValues<Nsd>& fe_values,
            FullMatrix<double>& Jlocal, Vector<double>& Rlocal,
            const std::vector<types::global_dof_index>& local_dof_indices
        );
        template<typename Number>
        void evaluate_fields(
            const FEValues<Nsd>& fe_values, const std::vector<Number>& local_dof_values, QPFields<Nsd,Number>& fields
        ) const;
        template<typename Number>
        void element_residual(
            const FEValues<Nsd>& fe_values, const QPFields<Nsd,Number>& fields, std::vector<Number>& Rlocal
        ) const;
        void solve();

        Triangulation<Nsd> triangulation;
        FESystem<Nsd> fe; //two variables per node
        DoFHandler<Nsd> dof_handler;
        unsigned int Nt;
        std::vector<FEValuesExtractors::Scalar> field_ext; //field_ext[f] picks fielf f

        SparsityPattern sparsity_pattern;
        SparseMatrix<double> NR_jacobian;
        Vector<double> sol_k, sol_n; //solution at current iterate, solution at previous timestep
        Vector<double> NR_update;
        Vector<double> NR_residual;

};
```

- Template parameters `Nsd` (dimension) and `BfOrder` (element order), as in all the user's codes.
- Parameters are stored as `const` members with a trailing underscore; `model_` and `output_writer_` are references to objects owned by `main()`.
- `evaluate_fields` and `element_residual` are *member function templates* on `Number`, so the same code runs with `ADNumber` (assembly) and `double`.
- Data: `fe` is the `FESystem` with two Q1 components; `field_ext[f]` are the extractors that pick field f; `sol_k` is the current Newton iterate, `sol_n` the converged solution at t^n.

```
template<unsigned int Nsd, unsigned int BfOrder>
PureMelt<Nsd,BfOrder>::PureMelt(
    const double x_ll, const double x_ul, const unsigned int n_elements, const unsigned int quadOrder,
    const unsigned int TEMP, const unsigned int PHI, const unsigned int N_FIELDS,
    const double undercooling, Point<Nsd> seed_center, const double seed_radius,
    const unsigned int NT, const double dt, const unsigned int N_NR, const double epsilon_NR,
    const ConstitutiveModel& model, OutputWriter<Nsd,BfOrder>& output_writer
) :
    x_ll_(x_ll), x_ul_(x_ul), n_elements_(n_elements), quadOrder_(quadOrder),
    TEMP_(TEMP), PHI_(PHI), N_FIELDS_(N_FIELDS),
```

```
    undercooling_(undercooling), seed_center_(seed_center), seed_radius_(
    seed_radius),
    NT_(NT), dt_(dt), N_NR_(N_NR), epsilon_NR_(epsilon_NR),
    model_(model),
    output_writer_(output_writer),
    fe(FE_Q<Nsd>(BfOrder), N_FIELDS_), //2 for 2 fields per node
    dof_handler(triangulation)
{
    for(unsigned int f = 0 ; f < N_FIELDS_ ; f++){
        field_ext.emplace_back(f);
    }
}

template<unsigned int Nsd, unsigned int BfOrder>
void PureMelt<Nsd,BfOrder>::run(){
    make_grid();
    setup_system();
    solve();
}
```

The constructor builds `fe(FE_Q<Nsd>(BfOrder), N_FIELDS_)` (two DoFs per node) and fills `field_ext` with one `FEValuesExtractors::Scalar` per field. `run()` is the top of the call tree: grid → setup → solve.

10.4 MakeGrid.tpp

`GridGenerator::subdivided_hyper_cube(triangulation, n_elements_, x_ll_, x_ul_)` creates $250 \times 250 = 62{,}500$ square cells of size $h = 2$ on $[-250, 250]^2$. The domain is centred on the origin so that a seed at $(0, 0)$ sits in the middle, matching the paper's 500×500 domain.

10.5 InitialConditions.hpp

```
template<unsigned int Nsd>
class InitialConditions : public Function<Nsd>{ //this class is the function in
    dealii's sense
    public: //everything public in function stays public in our class
        //class constructor
        InitialConditions(const double undercooling,
                          Point<Nsd>& seed_center,
                          const double seed_radius,
                          const double lambda0 = 1.0)//we fix lambda0 specifically
     for the initial condition seed here
        : Function<Nsd>(2), //2 for two fields - temp u and phi
          undercooling_(undercooling),
          seed_center_(seed_center),
          seed_radius_(seed_radius),
          lambda0_(lambda0)
        {}

        double value(const Point<Nsd>& p, const unsigned int component) const
    override{
            if(component == 0){ //component 0 is temperature u
                return undercooling_;
            }
```

```
            const double r = p.distance(seed_center_);
            return -std::tanh((r - seed_radius_)/(std::sqrt(2.0)*lambda0_)); //
    initialize phase field with a tanh profile of radius r about seed_center_
        }

    private:
        const double undercooling_; const Point<Nsd> seed_center_; const double
    seed_radius_, lambda0_;
};
```

- The class *is* a deal.II `Function` (inheritance), so `VectorTools::interpolate` can use it.
- `Function<Nsd>(2)` declares two components; it must match the FESystem.
- `value(p, component)` is called once per DoF: component 0 (u) returns Δ; component 1 (ϕ) returns $-\tanh\left((r-r_0)/\sqrt{2}\lambda_0\right)$ with $r = |\mathbf{p}-\mathbf{c}|$. The tanh profile is the 1D equilibrium of the double well ($\lambda_0^2\phi'' = \phi^3 - \phi$), so Newton does not have to repair a sharp jump (Fig. 5).

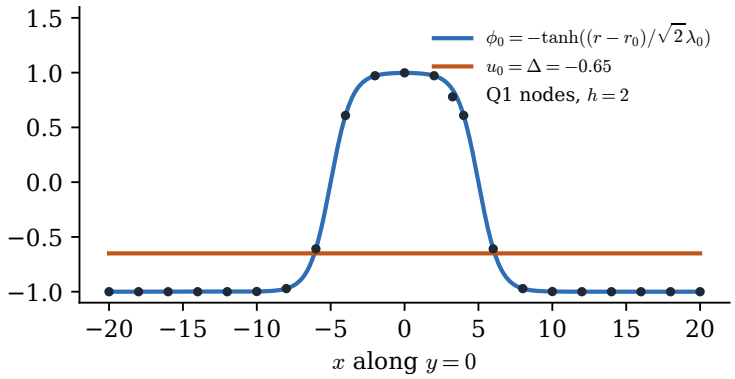


Figure 5: Initial condition along $y = 0$. Dots: nodal values on the $h = 2$ mesh.

10.6 SetupSystem.tpp

```
template<unsigned int Nsd, unsigned int BfOrder>
void PureMelt<Nsd,BfOrder>::setup_system(){
    dof_handler.distribute_dofs(fe);
    Nt = dof_handler.n_dofs();
    std::cout << "DoFs per vertex: " << fe.n_dofs_per_vertex() << std::endl;
    std::cout << "DoFs per cell: " << fe.n_dofs_per_cell() << std::endl;
    std::cout << "Total number of degrees of freedom: " << Nt << std::endl;

    DynamicSparsityPattern dsp(Nt);
    DoFTools::make_sparsity_pattern(dof_handler,dsp); //coupled both u and phi
    sparsity_pattern.copy_from(dsp);

    NR_jacobian.reinit(sparsity_pattern);
    NR_residual.reinit(Nt);
    sol_k.reinit(Nt);
    sol_n.reinit(Nt);
    NR_update.reinit(Nt);

```

```
    VectorTools::interpolate(dof_handler, InitialConditions<Nsd>(undercooling_,
    seed_center_, seed_radius_), sol_n);
    sol_k = sol_n; //guess for Newton's first iteration
}
```

Distributes the DoFs (2 per vertex, 8 per cell, 126,002 in total), builds a sparsity pattern that couples u and ϕ (both live on the same nodes), sizes all global vectors and the Jacobian, interpolates the initial condition into `sol_n`, and copies it into `sol_k` as the first Newton guess.

10.7 Solver.tpp — time loop and Newton loop

```
template <unsigned int Nsd, unsigned int BfOrder>
void PureMelt<Nsd,BfOrder>::solve(){

    std::cout << "Initiating solver..." << std::endl;

    double t = 0.0;

    std::ofstream pvd_output(output_writer_.get_output_dir() + "/final_solution.
    pvd");
    output_writer_.initiate_pvd(pvd_output); //initiated pvd file with headers
    output_writer_.write_vtu_and_pvd(dof_handler, sol_n, {"u", "phi"}, 0,
    pvd_output);

    for(unsigned int timestep = 1 ; timestep <= NT_ ; timestep++){
        t += dt_;
        sol_k = sol_n; //initial guess for the new time level

        //NR loops
        bool converged = false;
        for(unsigned int k = 0 ; k < N_NR_ ; k++){

            assemble_system(); //uses sol_k (and sol_n) -> NR_jacobian = dR/dd,
    NR_residual = R

            double error = NR_residual.l2_norm();
            std::cout << "timestep: " << timestep << " | iteration: " << k << " |
    residual: " << error << std::endl;

            if(error < epsilon_NR_){
                converged = true;
                std::cout << "convergence achieved for timestep " << timestep << "
     in " << k << " iterations" << std::endl;
                break;
            }

            //Solve J dd = -R
            NR_residual *= -1.0;
            NR_update = 0.0;
            SparseDirectUMFPACK directsolver;
            directsolver.initialize(NR_jacobian);
            directsolver.vmult(NR_update, NR_residual);

            sol_k += NR_update;
        }

        if(!converged){
            std::cout << "WARNING: Newton did not converge in " << N_NR_ << "
```

```
    iterations at timestep " << timestep << std::endl;
48          }
49
50          sol_n = sol_k; //accept the new time level
51
52          // if(timestep % 10 == 0){
53              output_writer_.write_vtu_and_pvd(dof_handler, sol_n, {"u", "phi"},
    timestep, pvd_output);
54          // }
55      }
56
57      output_writer_.finish_pvd(pvd_output); //finishes pvd file with footer
58
59      std::cout << "Congratulations! Solve Completed." << std::endl;
60  }
```

Lines	Action and theory
13–15	open the PVD file, write step 0
17–19	time loop; initial guess $\mathbf{d}^{n+1,(0)} = \mathbf{d}^n$
23–25	Newton loop; `assemble_system()` builds $\mathbf{J}$ and $\mathbf{R}$ at `sol_k`
27–34	convergence test $\|\mathbf{R}\|_2 < \varepsilon_{NR}$
37–41	solve $\mathbf{J}\,\Delta\mathbf{d} = -\mathbf{R}$ with UMFPACK (direct; $\mathbf{J}$ is non-symmetric)
43	$\mathbf{d}^{(k+1)} = \mathbf{d}^{(k)} + \Delta\mathbf{d}$
50–53	accept the step ($\mathbf{d}^n \leftarrow \mathbf{d}^{n+1}$) and write output

10.8 AssembleSystem.tpp — Level 1

```
5   template<unsigned int Nsd, unsigned int BfOrder>
6   void PureMelt<Nsd,BfOrder>::assemble_system(){
7       NR_jacobian = 0.0;
8       NR_residual = 0.0;
9
10      const QGauss<Nsd> quadrature_formula(quadOrder_);
11
12      // the following class handles three things at once - finite element(basis
        functions), quadrature, and mapping from parent to real domains
13      FEValues<Nsd> fe_values(fe, quadrature_formula, update_values |
        update_gradients | update_JxW_values);
14
15      const unsigned int dofs_per_cell = fe.n_dofs_per_cell();
16      std::vector<types::global_dof_index> local_dof_indices(dofs_per_cell);
17
18      FullMatrix<double> Jlocal(dofs_per_cell, dofs_per_cell);
19      Vector<double> Rlocal(dofs_per_cell);
20
21      for(const typename DoFHandler<Nsd>::active_cell_iterator &elem : dof_handler.
        active_cell_iterators()){
22          Jlocal = 0.0;
23          Rlocal = 0.0;
24
25          elem->get_dof_indices(local_dof_indices);
26          compute_element(elem, fe_values, Jlocal, Rlocal, local_dof_indices);
27
28          for(const unsigned int i : fe_values.dof_indices()){
```

```
29              NR_residual(local_dof_indices[i]) += Rlocal(i);
30              for(const unsigned int j : fe_values.dof_indices()){
31                  NR_jacobian.add(local_dof_indices[i], local_dof_indices[j], Jlocal
    (i,j));
32              }
33          }
34      }
35  }
```

Zeroes the global system, creates one `FEValues` (values, gradients, JxW) for all elements, and for each element calls `compute_element` and scatter-adds the local 8×8 matrix and 8-vector. "Add" rather than "copy" because neighbouring elements share nodes: $R_A = \sum_e R_A^{(e)}$, the integral split into a sum over elements. Nothing in this file knows about the physics or about AD.

10.9 ComputeElement.tpp — Level 2: AD on one element

```
7   template<unsigned int Nsd, unsigned int BfOrder>
8   void PureMelt<Nsd,BfOrder>::compute_element(
9       const typename DoFHandler<Nsd>::active_cell_iterator& elem, FEValues<Nsd>&
    fe_values,
10      FullMatrix<double>& Jlocal, Vector<double>& Rlocal,
11      const std::vector<types::global_dof_index>& local_dof_indices
12  ){
13      using namespace AD;
14
15      fe_values.reinit(elem);
16      const unsigned int dofs_per_cell = fe.n_dofs_per_cell();
17      const unsigned int n_q = fe_values.n_quadrature_points;
18
19      //Step 1: fresh helper for this element
20      ADHelper ad_helper(dofs_per_cell, dofs_per_cell);
21
22      //Step 2: register the current iterate as the independent variables
23      ad_helper.register_dof_values(sol_k, local_dof_indices); //pick out the cells
    values from global solution sol_k using local_dof_indices mapping
24      const std::vector<ADNumber>& dof_values_ad = ad_helper.
    get_sensitive_dof_values(); // sensitive = tracked
25
26      //Step 3: fields at the quadrature points, as AD Numbers
27      QPFields<Nsd,ADNumber> fields(N_FIELDS_, n_q);
28      evaluate_fields<ADNumber>(fe_values, dof_values_ad, fields);
29
30      //Step 4: compute the element residual as AD numbers
31      std::vector<ADNumber> residual_ad(dofs_per_cell, ADNumber(0.0)); //Creates the
     output: a vector with one entry per local DoF (8), each an AD number set to
    zero.
32      element_residual<ADNumber>(fe_values, fields, residual_ad);
33
34      //Step 5. read out R (values) and J (derivatives)
35      ad_helper.register_residual_vector(residual_ad);
36      ad_helper.compute_residual(Rlocal);
37      ad_helper.compute_linearization(Jlocal);
38  }
```

Steps 1–5 of Section 8: fresh helper (line 20), seeding from `sol_k` (23–24), fields at the quadrature points (27–28), residual (31–32), read-out of **R** and **J** (35–37). The file is identical for any problem.

10.10 ADConfig.hpp

```
#pragma once

namespace AD{
    // automatic differentiation setup
    constexpr Differentiation::AD::NumberTypes ADTypeCode = Differentiation::AD::NumberTypes::sacado_dfad;
    using ADHelper = Differentiation::AD::ResidualLinearization<ADTypeCode, double>;
    using ADNumber = typename ADHelper::ad_type;
}
```

The only place where the AD backend is chosen. Changing to another backend (e.g. `adolc_taped`) is a one-line change.

10.11 QPFields.hpp — the quadrature-point container

```
template<unsigned int Nsd, typename Number>
struct QPFields{//everything the residual needs at the quadrature point of each element
    QPFields(const unsigned int n_fields, const unsigned int n_q)
    :
        val(n_fields, std::vector<Number>(n_q)),
        grad(n_fields, std::vector<Tensor<1,Nsd,Number>>(n_q)), //Create a dynamic list of rank-1 tensors (each sized for Nsd dimensions using double precision), and size the list to hold n_q items
        val_old(n_fields, std::vector<double>(n_q)),
        grad_old(n_fields, std::vector<Tensor<1,Nsd,double>>(n_q))
    {}

    std::vector<std::vector<Number>>                val;
    std::vector<std::vector<Tensor<1,Nsd,Number>>>  grad;
    std::vector<std::vector<double>>                val_old;
    std::vector<std::vector<Tensor<1,Nsd,double>>>  grad_old;
};
```

A `struct` (public data) with two template parameters: the dimension `Nsd` and the number type `Number`. Each member is a table indexed `[field][quadrature point]`; `grad` entries are `Tensor<1,Nsd,Number>`. Old values are always `double` (data, not unknowns). It is sized by `n_fields`, so adding a field needs no change here.

10.12 EvaluateFields.tpp — Level 3a

```
template<unsigned int Nsd, unsigned int BfOrder>
template<typename Number>
void PureMelt<Nsd,BfOrder>::evaluate_fields(
    const FEValues<Nsd>& fe_values, const std::vector<Number>& local_dof_values,
    QPFields<Nsd,Number>& fields
) const {
    for(unsigned int f = 0; f < N_FIELDS_; f++){
        //current iterate : carries derivatives when Number = ADNumber
        fe_values[field_ext[f]].get_function_values_from_local_dof_values(local_dof_values, fields.val[f]);
        fe_values[field_ext[f]].get_function_gradients_from_local_dof_values(local_dof_values, fields.grad[f]);

```

```
13          //previous iterate : plain doubles
14          fe_values[field_ext[f]].get_function_values(sol_n, fields.val_old[f]);
15          fe_values[field_ext[f]].get_function_gradients(sol_n, fields.grad_old[f]);
16      }
17  }
```

For each field f, `fe_values[field_ext[f]]` is a *view* restricted to that component. Lines 10–11 compute $\phi_h(\mathbf{x}_q) = \sum_j N_j(\mathbf{x}_q) d_j$ and its gradient from the *local* AD values, so derivatives $\partial\phi_h/\partial d_j = N_j$ are carried along. Lines 14–15 read the old values from the *global* `sol_n` as doubles.

10.13 ElementResidual.tpp — Level 3b: the weak form

```
3   template<unsigned int Nsd, unsigned int BfOrder>
4   template<typename Number>
5   void PureMelt<Nsd,BfOrder>::element_residual(
6       const FEValues<Nsd>& fe_values, const QPFields<Nsd,Number>& fields, std::
        vector<Number>& Rlocal
7   ) const {
8       const double D = model_.diffusivity();
9
10      for(const unsigned int q : fe_values.quadrature_point_indices()){
11          //fields at this quadrature point
12          const Number&                u        = fields.val[TEMP_][q];
13          const Number&                phi      = fields.val[PHI_][q];
14          const Tensor<1,Nsd,Number>&  grad_u   = fields.grad[TEMP_][q];
15          const Tensor<1,Nsd,Number>&  grad_phi = fields.grad[PHI_][q];
16          const double                 u_n      = fields.val_old[TEMP_][q];
17          const double                 phi_n    = fields.val_old[PHI_][q];
18
19          //nonlinear constitutive quantities
20          Number lambda, tau;
21          Tensor<1,Nsd,Number> dlambda_dq;
22          model_.interface_coefficients(grad_phi, lambda, dlambda_dq, tau);
23
24          const Number dfdphi   = model_.bulk_driving_force(phi, u);
25          const Number hprime   = model_.latent_heat_slope(phi);
26
27          //time derivatives (backward Euler)
28          const Number dudt     = (u - u_n)/dt_;
29          const Number dphidt   = (phi - phi_n)/dt_;
30
31          //gradient + anisotropy flux: lambda^2 grad phi + |grad phi|^2 lambda
        dlambda/dq
32          const Number grad_phi2 = grad_phi*grad_phi;
33          const Number lambda2   = lambda*lambda;
34          const Number coef      = grad_phi2*lambda;
35          const Tensor<1,Nsd,Number> phi_flux = lambda2*grad_phi + coef*dlambda_dq;
36
37          const double JxW = fe_values.JxW(q);
38
39          for(const unsigned int i : fe_values.dof_indices()){
40              //shape functions of each field (zero if dof i belongs to another
        field)
41              const double          N_u       = fe_values[field_ext[TEMP_]].value(i,
        q);
42              const Tensor<1,Nsd>   gradN_u   = fe_values[field_ext[TEMP_]].gradient
        (i,q);
43              const double          N_phi     = fe_values[field_ext[PHI_]].value(i,q
```

```
    );
44              const Tensor<1,Nsd>   gradN_phi = fe_values[field_ext[PHI_]].gradient(
    i,q);
45
46              //Eq. 6a : temperature
47              const Number R_u   = N_u*(dudt - 0.5*hprime*dphidt) + D*(gradN_u*
    grad_u);
48              //Eq. 6b : phase field
49              const Number R_phi = N_phi*(tau*dphidt + dfdphi) + gradN_phi*phi_flux;
50
51              Rlocal[i] += (R_u + R_phi)*JxW;
52          }
53      }
54  }
```

This is the only file containing the PDEs. It implements Section 7 line by line:

Lines	Code	Theory
10	loop over quadrature points	$\sum_q$
12–17	read $u_h, \phi_h, \nabla u_h, \nabla \phi_h$ (`Number`) and u_h^n, ϕ_h^n (`double`)	fields at $\mathbf{x}_q$
20–22	`interface_coefficients`	$\lambda, \partial\lambda/\partial\mathbf{q}, \tau$
24	`bulk_driving_force`	$\partial f/\partial\phi$
25	`latent_heat_slope`	$h'(\phi)$
28–29	backward Euler differences	$(u-u^n)/\Delta t$, $(\phi-\phi^n)/\Delta t$
32–35	`phi_flux`	$\boldsymbol{\Theta} = \lambda^2\nabla\phi + \lvert\nabla\phi\rvert^2\lambda\,\partial\lambda/\partial\mathbf{q}$
37	`JxW`	quadrature weight $\times$ Jacobian
39–44	test functions of each field (zero for the other field)	$N_A, \nabla N_A$
47	`R_u`	$N_A\, s_u + \nabla N_A \cdot \mathbf{v}_u \;\; (T_1+T_2+T_3)$
49	`R_phi`	$N_A\, s_\phi + \nabla N_A \cdot \mathbf{v}_\phi \;\; (T_4+T_5+T_6)$
51	`Rlocal[i] += (R_u + R_phi)*JxW`	quadrature sum

One loop, two equations. Each local DoF i belongs to one field. For a u-DoF the ϕ shape functions are zero, so only `R_u` survives (Eq. 6a); for a ϕ-DoF only `R_phi` (Eq. 6b). Types follow one rule: a quantity that depends on the unknowns at t^{n+1} is `Number`; geometry, old values and parameters are `double`.

10.14 ConstitutiveModel.hpp — the nonlinear laws

```
3   class ConstitutiveModel{
4       public:
5           ConstitutiveModel(const double D, const double eps4, const double lambda0,
6                             const double tau0, const double xi)
7           : D_(D), eps4_(eps4), lambda0_(lambda0), tau0_(tau0), xi_(xi) {}
8
9           double diffusivity() const { return D_; }
10
11          //bulk driving force  df/dphi = g'(phi) + xi u p'(phi) = -phi + phi^3 + xi
     u (1 - phi^2)^2
12          template <typename T>
13          T bulk_driving_force(const T& phi, const T& u) const{
14              const T one_m_phi2 = 1.0 - phi*phi;
```

```
            return -phi + phi*phi*phi + xi_*u*one_m_phi2*one_m_phi2;
        }

        //latent heat function h(phi) and its slope h'(phi) (U = u - h(phi)/2); here h = phi
        template <typename T>
        T latent_heat_slope(const T& /*phi*/) const{ return T(1.0); }

        //four-fold anisotropy a_s(q) = (1-3eps4) + 4 eps4 (qx^4+qy^4)/|q|^4, q = grad phi
        //returns a_s and d a_s / d q
        template <int dim, typename T>
        void anisotropy(const Tensor<1,dim,T>& q, T& a, Tensor<1,dim,T>& da_dq) const{
            static_assert(dim == 2, "anisotropy() is written for 2D");
            const double delta = 1e-12; //regularisation where grad phi = 0 (bulk)
            const T qx = q[0], qy = q[1];
            const T q2 = qx*qx + qy*qy + delta;
            const T q4 = q2*q2;
            const T q6 = q4*q2;
            const T S  = (qx*qx*qx*qx + qy*qy*qy*qy)/q4;
            a = (1.0 - 3.0*eps4_) + 4.0*eps4_*S;
            da_dq[0] = 4.0*eps4_*(4.0*qx*qy*qy*(qx*qx - qy*qy)/q6);
            da_dq[1] = 4.0*eps4_*(4.0*qx*qx*qy*(qy*qy - qx*qx)/q6);
        }

        //interface coefficients from the anisotropy:
        //lambda(n) = lambda0 a_s,  d lambda / d q,  tau(n) = tau0 a_s^2
        template <int dim, typename T>
        void interface_coefficients(const Tensor<1,dim,T>& q, T& lambda, Tensor<1,dim,T>& dlambda_dq, T& tau) const{
            T a;
            Tensor<1,dim,T> da_dq;
            anisotropy(q, a, da_dq);
            lambda     = lambda0_*a;
            dlambda_dq = lambda0_*da_dq;
            tau        = tau0_*a*a;
        }

    private:
        const double D_, eps4_, lambda0_, tau0_, xi_;
};
```

Function	Formula
`diffusivity()`	D
`bulk_driving_force(phi,u)`	$\partial f/\partial\phi = -\phi + \phi^3 + \xi u(1-\phi^2)^2$
`latent_heat_slope(phi)`	$h'(\phi) = 1$ (parameter unnamed because unused)
`anisotropy(q,a,da_dq)`	$a_s = (1-3\epsilon_4) + 4\epsilon_4 S$, $\partial a_s/\partial\mathbf{q} = 4\epsilon_4\,\partial S/\partial\mathbf{q}$
`interface_coefficients(q,lambda,dlambda_dq,tau)`	$\lambda = \lambda_0 a_s$, $\partial\lambda/\partial\mathbf{q} = \lambda_0\partial a_s/\partial\mathbf{q}$, $\tau = \tau_0 a_s^2$

Every function is a template on the number type `T`; called with `ADNumber` the derivatives flow through it. Only the functions in the residual are written; their derivatives (e.g. $\partial^2 f/\partial\phi^2$, sec-

ond derivatives of a_s) come from AD. Functions with several results write them through non-const reference parameters. The template parameter `int dim` (not `unsigned`) matches deal.II's `Tensor<rank,int dim,Number>`.

10.15 OutputWriter.hpp/.tpp

Writes one VTU file per call with one named data vector per FESystem component (`{"u", "phi"}`) and appends a `DataSet` entry to a PVD collection (`output/final_solution.pvd`) that ParaView opens as a time series.

11 Results

11.1 Dendrite evolution

Figure 6 shows frames of the simulation output: the phase field (top) and temperature (bottom). The circular seed develops four arms along the x and y axes — the directions of maximum a_s — and side branches appear behind the tips. Latent heat released at the interface warms the solid towards $u \approx 0$; a thin thermal boundary layer surrounds the arms, and the tips, which reach into the coldest melt, grow fastest.

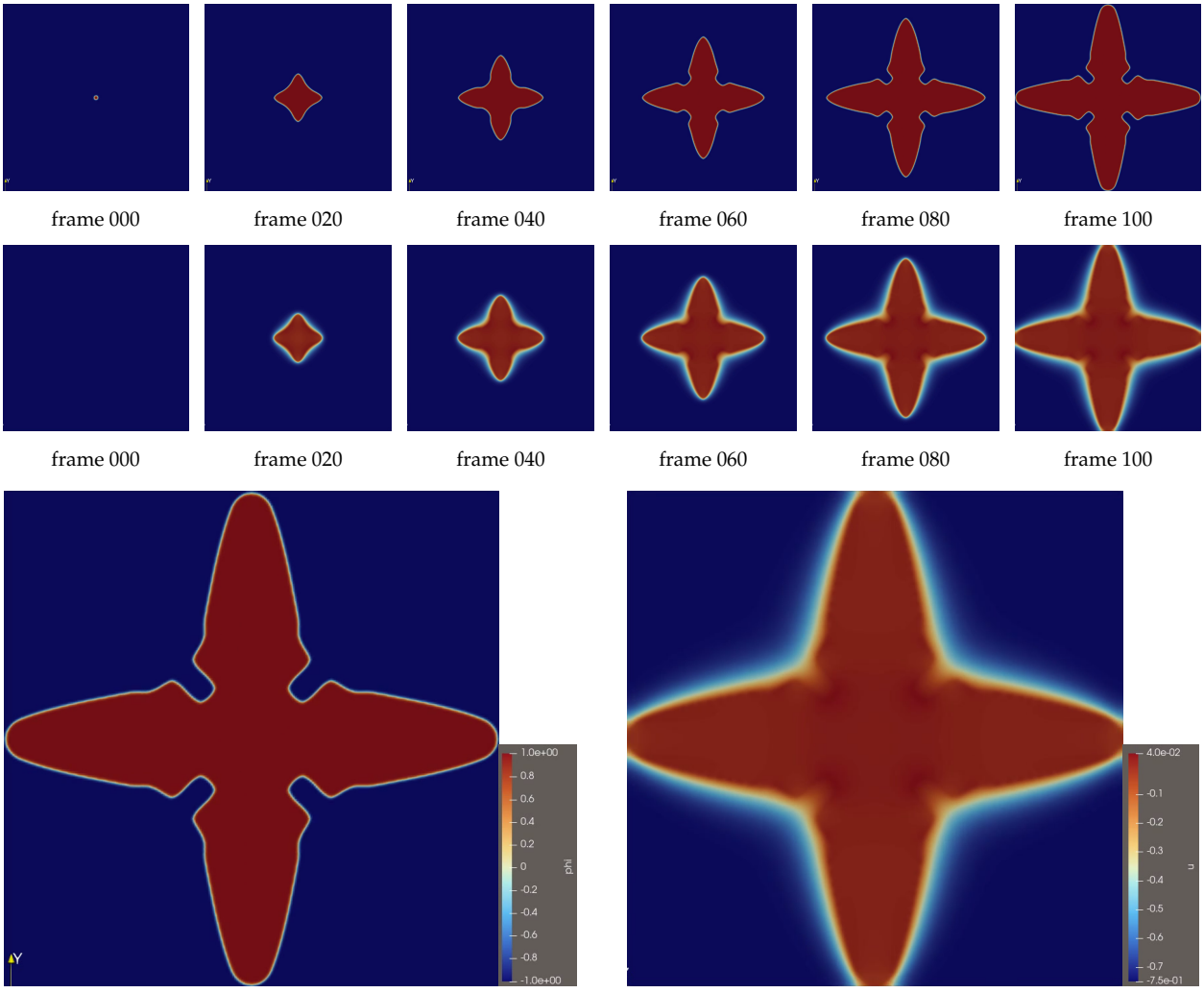
11.2 Animation

The full animation (101 frames, ϕ left and u right) is embedded in this PDF as a file attachment: **simulation.mp4** (click to open). In Adobe Acrobat or Reader, click the link or open the attachments panel; in other viewers, use the viewer's attachment list. The repository's main.cpp currently sets `NT = 100`, `dt = 0.01` ($t_{\text{end}} = 1$); the run settings used for the animation are not recorded in the repository and should be noted alongside it.

11.3 Newton convergence

To document solver behaviour, the code was compiled unchanged (except for the deal.II version and path in CMakeLists.txt) with deal.II 9.5.1 and run for three time steps with the repository parameters:

```
Number of active cells: 62500
DoFs per vertex: 2
DoFs per cell: 8
Total number of degrees of freedom: 126002
timestep: 1 | iteration: 0 | residual: 10.4678
timestep: 1 | iteration: 1 | residual: 0.00339069
timestep: 1 | iteration: 2 | residual: 0.000465251
timestep: 1 | iteration: 3 | residual: 0.000152855
timestep: 1 | iteration: 4 | residual: 7.07204e-06
timestep: 1 | iteration: 5 | residual: 6.74902e-09
timestep: 2 | iteration: 0 | residual: 10.3953
timestep: 2 | iteration: 1 | residual: 0.00341735
timestep: 2 | iteration: 2 | residual: 1.36277e-06
timestep: 2 | iteration: 3 | residual: 5.07605e-12
timestep: 3 | iteration: 0 | residual: 10.3247
timestep: 3 | iteration: 1 | residual: 0.00274252
```



(a) ϕ, final frame **(b)** u, final frame

Figure 6: Dendrite growth into an undercooled melt ($\Delta = -0.65$, $\epsilon_4 = 0.05$) on $[-250, 250]^2$. Top row: ϕ (-1 liquid, blue; $+1$ solid, red). Second row: u (range -0.75 to 0.04). Frames from the attached animation.

```
timestep: 3 | iteration: 2 | residual: 1.70458e-07
```

From step 2 on, convergence is quadratic (correct digits roughly double per iteration), the signature of an exact Jacobian. The first step needs five iterations: the continuous tanh seed is not the discrete equilibrium on the $h = 2$ mesh, so the first solve also relaxes the interface profile. A step took about 18 s on a single core (three assemblies plus UMFPACK factorisations of a 126,002-unknown system).

12 Notes and possible extensions

- **Interface resolution.** The mesh size is $h = 2$ while the interface width is $\lambda_0 = 1$, so the diffuse interface spans only one to two elements. Karma & Rappel converged their tip velocity at $\Delta x/W_0 = 0.4$ (with a finite-difference scheme), and the paper studies convergence under refinement. A refinement study (e.g. $h = 1, 0.5$) of the tip velocity against the Green's function value $v_{\text{tip}} = 0.0469$ (Bhagat Sect. 4.1) would quantify the resolution error; adaptive refinement near $|\phi| < 1$ makes this affordable.
- **Jacobian check.** Solver.tpp has `VerifyJacobian.tpp` commented out. Re-enabling a finite-difference comparison on a few interface elements after any change to ElementResidual.tpp or ConstitutiveModel.hpp catches errors early (relative differences $\sim 10^{-8}$ are expected).
- **Tolerance.** `epsilon_NR` is absolute; a relative criterion $\|\mathbf{R}\| < \varepsilon\|\mathbf{R}_0\|$ is independent of mesh size and domain.
- **Small robustness edits.** `Function<Nsd>(2)` in InitialConditions.hpp could use the number of fields instead of the literal 2; its constructor can take `const Point<Nsd>&`. Writing output every step produces 101 files of 126,002 DoFs; an output interval keeps disk use down for long runs.
- **Extensions.** A new field (e.g. solute concentration for the binary-alloy model, Bhagat Sect. 2.3) needs a new component index, `N_FIELDS = 3`, its equation in ElementResidual.tpp, and any new laws in ConstitutiveModel.hpp; the AD layer, QPFields.hpp and EvaluateFields.tpp adapt unchanged.

References

[1] K. Bhagat, S. Rudraraju, Modeling of dendritic solidification and numerical analysis of the phase-field approach to model complex morphologies in alloys, *Engineering with Computers* 39 (2023) 2345–2363.

[2] A. Karma, W.-J. Rappel, Quantitative phase-field modeling of dendritic growth in two and three dimensions, *Phys. Rev. E* 57 (1998) 4323. doi:10.1103/PhysRevE.57.4323

[3] M. Plapp, A. Karma, Multiscale finite-difference-diffusion-Monte-Carlo method for simulating dendritic solidification, *J. Comput. Phys.* 165 (2000) 592–619.

[4] W. J. Boettinger, J. A. Warren, C. Beckermann, A. Karma, Phase-field simulation of solidification, *Annu. Rev. Mater. Res.* 32 (2002) 163–194.

[5] deal.II tutorial step-72 and the `Differentiation::AD` module documentation (automatic differentiation with `ResidualLinearization`).

A Full source listings

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(PureMelt CXX)

# Force your preferred build type here (e.g., "Debug" or "Release")
set(CMAKE_BUILD_TYPE "Release" CACHE STRING "Choose the build type" FORCE)

# Point CMake to your deal.II installation path
find_package(deal.II 9.7.1 REQUIRED
  HINTS /home/grathod/Downloads/dealii/dealii/installation_files
#   HINTS $ENV{DEAL_II_DIR}
)

include_directories("include")

# Initialize cached variables required by deal.II
DEAL_II_INITIALIZE_CACHED_VARIABLES()

# release
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -O3 -g -march=native")

#debug
# set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -fsanitize=address -g -march=native")

# Define your executable and source file
add_executable(main main.cpp)

# Link deal.II and all dependencies (MPI, PETSc, Trilinos, p4est)
DEAL_II_SETUP_TARGET(main)
```

main.cpp

```
// Pure melt dendrite evolution (Implicit]) - KBhagat 2023

#include <deal.II/grid/tria.h> //triangulation
#include <deal.II/dofs/dof_handler.h> //enumeration of degrees of freedom
#include <deal.II/dofs/dof_tools.h>
#include <deal.II/grid/grid_generator.h> //grid generation
#include <deal.II/base/point.h>
#include <deal.II/numerics/vector_tools.h>

#include <deal.II/fe/fe_system.h>
#include <deal.II/fe/fe_q.h>
#include <deal.II/fe/fe_values_extractors.h>

#include <deal.II/fe/fe_values.h> //used to assemble matrix using quadrature on
    each cell
#include <deal.II/base/quadrature_lib.h> //quadrature rules

#include <deal.II/differentiation/ad.h>

//linear algebra
#include <deal.II/lac/vector.h>
#include <deal.II/lac/full_matrix.h>
```

```
#include <deal.II/lac/sparse_matrix.h>
#include <deal.II/lac/dynamic_sparsity_pattern.h>
#include <deal.II/lac/sparse_direct.h>

//output
#include <deal.II/numerics/data_out.h>
#include <fstream>
#include <iostream>
#include <filesystem>
#include <cmath>

using namespace dealii;
#include "PureMelt.hpp"
#include "ConstitutiveModel.hpp"
#include "OutputWriter.hpp"

int main(){
    constexpr unsigned int Nsd = 2;
    constexpr unsigned int BfOrder = 1;
    const unsigned int quadOrder = BfOrder + 1;

    //Mesh
    const double x_ll = -250.0;
    const double x_ul = 250.0;
    const unsigned int n_elements = 250; //number of elements per side

    //NR parameters
    const unsigned int NT = 100; //total number of timesteps
    const double dt = 0.01; //timestep size
    const unsigned int N_NR = 10; //total number of newton iterates per timestep
    const double epsilon_NR = 1e-6; //NR convergence tolerance

    //Field variables of the problem (component index inside FESystem)
    static constexpr unsigned int TEMP = 0; //u : scaled temperature
    static constexpr unsigned int PHI = 1; //phi : phase field order parameter
    static constexpr unsigned int N_FIELDS = 2; //total two fields in this problem

    //Initial conditions
    const double undercooling = -0.65;
    Point<Nsd> seed_center;
    seed_center[0] = 0.0; seed_center[1] = 0.0; //seed_center[2] = 0.0;
    const double seed_radius = 5.0;

    const double D = 1.0;
    const double eps4 = 0.05;
    const double lambda0 = 1.0;
    const double tau0 = 1.0;
    const double xi = 1.596;
    ConstitutiveModel model(D, eps4, lambda0, tau0, xi);

    OutputWriter<Nsd,BfOrder> output_writer("output");

    PureMelt<Nsd,BfOrder> problem(
        x_ll, x_ul, n_elements, quadOrder,
        TEMP, PHI, N_FIELDS, undercooling, seed_center, seed_radius,
        NT, dt, N_NR, epsilon_NR,
        model, output_writer
    );

```

```
    problem.run();

}
```

PureMelt.hpp

```
#pragma once

#include "OutputWriter.hpp"
#include "QPFields.hpp"

#include "ConstitutiveModel.hpp"

template<unsigned int Nsd, unsigned int BfOrder>
class PureMelt{
    public:
        PureMelt(
            const double x_ll, const double x_ul, const unsigned int n_elements,
    const unsigned int quadOrder, //mesh
            const unsigned int TEMP, const unsigned int PHI, const unsigned int
    N_FIELDS,
            const double undercooling, Point<Nsd> seed_center, const double
    seed_radius, //initial conditions
            const unsigned int NT, const double dt, const unsigned int N_NR, const
     double epsilon_NR, //newton solver
            const ConstitutiveModel& model, OutputWriter<Nsd,BfOrder>&
    output_writer
        );
        void run();

    private:
        const double x_ll_, x_ul_;
        const unsigned int n_elements_;
        const unsigned int quadOrder_;
        const unsigned int TEMP_;
        const unsigned int PHI_;
        const unsigned int N_FIELDS_;
        const double undercooling_;
        Point<Nsd> seed_center_;
        const double seed_radius_;
        const unsigned int NT_;
        const double dt_;
        const unsigned int N_NR_;
        const double epsilon_NR_;
        const ConstitutiveModel& model_;
        OutputWriter<Nsd,BfOrder>& output_writer_;

        void make_grid();
        void setup_system();
        void assemble_system();
        void compute_element(
            const typename DoFHandler<Nsd>::active_cell_iterator& elem, FEValues<
    Nsd>& fe_values,
            FullMatrix<double>& Jlocal, Vector<double>& Rlocal,
            const std::vector<types::global_dof_index>& local_dof_indices
        );
        template<typename Number>
        void evaluate_fields(
```

```
47              const FEValues<Nsd>& fe_values, const std::vector<Number>&
    local_dof_values, QPFields<Nsd,Number>& fields
48          ) const;
49          template<typename Number>
50          void element_residual(
51              const FEValues<Nsd>& fe_values, const QPFields<Nsd,Number>& fields,
    std::vector<Number>& Rlocal
52          ) const;
53          void solve();
54
55          Triangulation<Nsd> triangulation;
56          FESystem<Nsd> fe; //two variables per node
57          DoFHandler<Nsd> dof_handler;
58          unsigned int Nt;
59          std::vector<FEValuesExtractors::Scalar> field_ext; //field_ext[f] picks
    fielf f
60
61          SparsityPattern sparsity_pattern;
62          SparseMatrix<double> NR_jacobian;
63          Vector<double> sol_k, sol_n; //solution at current iterate, solution at
    previous timestep
64          Vector<double> NR_update;
65          Vector<double> NR_residual;
66
67  };
68
69  #include "PureMelt.tpp"
70  #include "MakeGrid.tpp"
71  #include "SetupSystem.tpp"
72  #include "Solver.tpp"
```

PureMelt.tpp

```
1   #pragma once
2
3   template<unsigned int Nsd, unsigned int BfOrder>
4   PureMelt<Nsd,BfOrder>::PureMelt(
5       const double x_ll, const double x_ul, const unsigned int n_elements, const
        unsigned int quadOrder,
6       const unsigned int TEMP, const unsigned int PHI, const unsigned int N_FIELDS,
7       const double undercooling, Point<Nsd> seed_center, const double seed_radius,
8       const unsigned int NT, const double dt, const unsigned int N_NR, const double
        epsilon_NR,
9       const ConstitutiveModel& model, OutputWriter<Nsd,BfOrder>& output_writer
10  ) :
11      x_ll_(x_ll), x_ul_(x_ul), n_elements_(n_elements), quadOrder_(quadOrder),
12      TEMP_(TEMP), PHI_(PHI), N_FIELDS_(N_FIELDS),
13      undercooling_(undercooling), seed_center_(seed_center), seed_radius_(
        seed_radius),
14      NT_(NT), dt_(dt), N_NR_(N_NR), epsilon_NR_(epsilon_NR),
15      model_(model),
16      output_writer_(output_writer),
17      fe(FE_Q<Nsd>(BfOrder), N_FIELDS_), //2 for 2 fields per node
18      dof_handler(triangulation)
19  {
20      for(unsigned int f = 0 ; f < N_FIELDS_ ; f++){
21          field_ext.emplace_back(f);
22      }
```

```
}

template<unsigned int Nsd, unsigned int BfOrder>
void PureMelt<Nsd,BfOrder>::run(){
    make_grid();
    setup_system();
    solve();
}
```

MakeGrid.tpp

```
#pragma once

template <unsigned int Nsd, unsigned int BfOrder>
void PureMelt<Nsd,BfOrder>::make_grid(){
    GridGenerator::subdivided_hyper_cube(triangulation, n_elements_, x_ll_, x_ul_);
    std::cout << "Number of active cells: " << triangulation.n_active_cells() << std::endl;
}
```

InitialConditions.hpp

```
#pragma once

template<unsigned int Nsd>
class InitialConditions : public Function<Nsd>{ //this class is the function in dealii's sense
    public: //everything public in function stays public in our class
        //class constructor
        InitialConditions(const double undercooling,
                          Point<Nsd>& seed_center,
                          const double seed_radius,
                          const double lambda0 = 1.0)//we fix lambda0 specifically for the initial condition seed here
        : Function<Nsd>(2), //2 for two fields - temp u and phi
          undercooling_(undercooling),
          seed_center_(seed_center),
          seed_radius_(seed_radius),
          lambda0_(lambda0)
        {}

        double value(const Point<Nsd>& p, const unsigned int component) const override{
            if(component == 0){ //component 0 is temperature u
                return undercooling_;
            }
            const double r = p.distance(seed_center_);
            return -std::tanh((r - seed_radius_)/(std::sqrt(2.0)*lambda0_)); //initialize phase field with a tanh profile of radius r about seed_center_
        }

    private:
        const double undercooling_; const Point<Nsd> seed_center_; const double seed_radius_, lambda0_;
};
```

SetupSystem.tpp

```
#pragma once

#include "InitialConditions.hpp"

template<unsigned int Nsd, unsigned int BfOrder>
void PureMelt<Nsd,BfOrder>::setup_system(){
    dof_handler.distribute_dofs(fe);
    Nt = dof_handler.n_dofs();
    std::cout << "DoFs per vertex: " << fe.n_dofs_per_vertex() << std::endl;
    std::cout << "DoFs per cell: " << fe.n_dofs_per_cell() << std::endl;
    std::cout << "Total number of degrees of freedom: " << Nt << std::endl;

    DynamicSparsityPattern dsp(Nt);
    DoFTools::make_sparsity_pattern(dof_handler,dsp); //coupled both u and phi
    sparsity_pattern.copy_from(dsp);

    NR_jacobian.reinit(sparsity_pattern);
    NR_residual.reinit(Nt);
    sol_k.reinit(Nt);
    sol_n.reinit(Nt);
    NR_update.reinit(Nt);

    VectorTools::interpolate(dof_handler, InitialConditions<Nsd>(undercooling_,
    seed_center_, seed_radius_), sol_n);
    sol_k = sol_n; //guess for Newton's first iteration
}
```

Solver.tpp

```
#pragma once

#include "AssembleSystem.tpp"
// #include "VerifyJacobian.tpp"

template <unsigned int Nsd, unsigned int BfOrder>
void PureMelt<Nsd,BfOrder>::solve(){

    std::cout << "Initiating solver..." << std::endl;

    double t = 0.0;

    std::ofstream pvd_output(output_writer_.get_output_dir() + "/final_solution.
    pvd");
    output_writer_.initiate_pvd(pvd_output); //initiated pvd file with headers
    output_writer_.write_vtu_and_pvd(dof_handler, sol_n, {"u", "phi"}, 0,
    pvd_output);

    for(unsigned int timestep = 1 ; timestep <= NT_ ; timestep++){
        t += dt_;
        sol_k = sol_n; //initial guess for the new time level

        //NR loops
        bool converged = false;
        for(unsigned int k = 0 ; k < N_NR_ ; k++){

            assemble_system(); //uses sol_k (and sol_n) -> NR_jacobian = dR/dd,
```

```
    NR_residual = R

            double error = NR_residual.l2_norm();
            std::cout << "timestep: " << timestep << " | iteration: " << k << " | residual: " << error << std::endl;

            if(error < epsilon_NR_){
                converged = true;
                std::cout << "convergence achieved for timestep " << timestep << " in " << k << " iterations" << std::endl;
                break;
            }

            //Solve J dd = -R
            NR_residual *= -1.0;
            NR_update = 0.0;
            SparseDirectUMFPACK directsolver;
            directsolver.initialize(NR_jacobian);
            directsolver.vmult(NR_update, NR_residual);

            sol_k += NR_update;
        }

        if(!converged){
            std::cout << "WARNING: Newton did not converge in " << N_NR_ << " iterations at timestep " << timestep << std::endl;
        }

        sol_n = sol_k; //accept the new time level

        // if(timestep % 10 == 0){
            output_writer_.write_vtu_and_pvd(dof_handler, sol_n, {"u", "phi"}, timestep, pvd_output);
        // }
    }

    output_writer_.finish_pvd(pvd_output); //finishes pvd file with footer

    std::cout << "Congratulations! Solve Completed." << std::endl;
}
```

AssembleSystem.tpp

```
#pragma once

#include "ComputeElement.tpp"

template<unsigned int Nsd, unsigned int BfOrder>
void PureMelt<Nsd,BfOrder>::assemble_system(){
    NR_jacobian = 0.0;
    NR_residual = 0.0;

    const QGauss<Nsd> quadrature_formula(quadOrder_);

    // the following class handles three things at once - finite element(basis functions), quadrature, and mapping from parent to real domains
    FEValues<Nsd> fe_values(fe, quadrature_formula, update_values | update_gradients | update_JxW_values);
```

```
14
15      const unsigned int dofs_per_cell = fe.n_dofs_per_cell();
16      std::vector<types::global_dof_index> local_dof_indices(dofs_per_cell);
17
18      FullMatrix<double> Jlocal(dofs_per_cell, dofs_per_cell);
19      Vector<double> Rlocal(dofs_per_cell);
20
21      for(const typename DoFHandler<Nsd>::active_cell_iterator &elem : dof_handler.
        active_cell_iterators()){
22          Jlocal = 0.0;
23          Rlocal = 0.0;
24
25          elem->get_dof_indices(local_dof_indices);
26          compute_element(elem, fe_values, Jlocal, Rlocal, local_dof_indices);
27
28          for(const unsigned int i : fe_values.dof_indices()){
29              NR_residual(local_dof_indices[i]) += Rlocal(i);
30              for(const unsigned int j : fe_values.dof_indices()){
31                  NR_jacobian.add(local_dof_indices[i], local_dof_indices[j], Jlocal
        (i,j));
32              }
33          }
34      }
35  }
```

ComputeElement.tpp

```
1   #pragma once
2
3   #include "ADConfig.hpp"
4   #include "EvaluateFields.tpp"
5   #include "ElementResidual.tpp"
6
7   template<unsigned int Nsd, unsigned int BfOrder>
8   void PureMelt<Nsd,BfOrder>::compute_element(
9       const typename DoFHandler<Nsd>::active_cell_iterator& elem, FEValues<Nsd>&
        fe_values,
10      FullMatrix<double>& Jlocal, Vector<double>& Rlocal,
11      const std::vector<types::global_dof_index>& local_dof_indices
12  ){
13      using namespace AD;
14
15      fe_values.reinit(elem);
16      const unsigned int dofs_per_cell = fe.n_dofs_per_cell();
17      const unsigned int n_q = fe_values.n_quadrature_points;
18
19      //Step 1: fresh helper for this element
20      ADHelper ad_helper(dofs_per_cell, dofs_per_cell);
21
22      //Step 2: register the current iterate as the independent variables
23      ad_helper.register_dof_values(sol_k, local_dof_indices); //pick out the cells
        values from global solution sol_k using local_dof_indices mapping
24      const std::vector<ADNumber>& dof_values_ad = ad_helper.
        get_sensitive_dof_values(); // sensitive = tracked
25
26      //Step 3: fields at the quadrature points, as AD Numbers
27      QPFields<Nsd,ADNumber> fields(N_FIELDS_, n_q);
28      evaluate_fields<ADNumber>(fe_values, dof_values_ad, fields);
```

```
29
30      //Step 4: compute the element residual as AD numbers
31      std::vector<ADNumber> residual_ad(dofs_per_cell, ADNumber(0.0)); //Creates the
         output: a vector with one entry per local DoF (8), each an AD number set to
        zero.
32      element_residual<ADNumber>(fe_values, fields, residual_ad);
33
34      //Step 5. read out R (values) and J (derivatives)
35      ad_helper.register_residual_vector(residual_ad);
36      ad_helper.compute_residual(Rlocal);
37      ad_helper.compute_linearization(Jlocal);
38  }
```

ADConfig.hpp

```
1   #pragma once
2
3   namespace AD{
4       // automatic differentiation setup
5       constexpr Differentiation::AD::NumberTypes ADTypeCode = Differentiation::AD::
        NumberTypes::sacado_dfad;
6       using ADHelper = Differentiation::AD::ResidualLinearization<ADTypeCode, double
        >;
7       using ADNumber = typename ADHelper::ad_type;
8   }
```

QPFields.hpp

```
1   #pragma once
2
3   // QPFields is a container. for one element, it holds every field's value and
        gradient at every quadrature point, at the new time level and at the old one.
4   // evaluate_fields() fills it, and element_residual() reads it.
5
6   template<unsigned int Nsd, typename Number>
7   struct QPFields{//everything the residual needs at the quadrature point of each
        element
8       QPFields(const unsigned int n_fields, const unsigned int n_q)
9       :
10          val(n_fields, std::vector<Number>(n_q)),
11          grad(n_fields, std::vector<Tensor<1,Nsd,Number>>(n_q)), //Create a dynamic
         list of rank-1 tensors (each sized for Nsd dimensions using double precision)
        , and size the list to hold n_q items
12          val_old(n_fields, std::vector<double>(n_q)),
13          grad_old(n_fields, std::vector<Tensor<1,Nsd,double>>(n_q))
14      {}
15
16      std::vector<std::vector<Number>>                val;
17      std::vector<std::vector<Tensor<1,Nsd,Number>>>  grad;
18      std::vector<std::vector<double>>                val_old;
19      std::vector<std::vector<Tensor<1,Nsd,double>>>  grad_old;
20  };
```

EvaluateFields.tpp

```
1   #pragma once
2
```

```
template<unsigned int Nsd, unsigned int BfOrder>
template<typename Number>
void PureMelt<Nsd,BfOrder>::evaluate_fields(
    const FEValues<Nsd>& fe_values, const std::vector<Number>& local_dof_values,
    QPFields<Nsd,Number>& fields
) const {
    for(unsigned int f = 0; f < N_FIELDS_; f++){
        //current iterate : carries derivatives when Number = ADNumber
        fe_values[field_ext[f]].get_function_values_from_local_dof_values(
    local_dof_values, fields.val[f]);
        fe_values[field_ext[f]].get_function_gradients_from_local_dof_values(
    local_dof_values, fields.grad[f]);

        //previous iterate : plain doubles
        fe_values[field_ext[f]].get_function_values(sol_n, fields.val_old[f]);
        fe_values[field_ext[f]].get_function_gradients(sol_n, fields.grad_old[f]);
    }
}
```

ElementResidual.tpp

```
#pragma once

template<unsigned int Nsd, unsigned int BfOrder>
template<typename Number>
void PureMelt<Nsd,BfOrder>::element_residual(
    const FEValues<Nsd>& fe_values, const QPFields<Nsd,Number>& fields, std::
    vector<Number>& Rlocal
) const {
    const double D = model_.diffusivity();

    for(const unsigned int q : fe_values.quadrature_point_indices()){
        //fields at this quadrature point
        const Number&                u        = fields.val[TEMP_][q];
        const Number&                phi      = fields.val[PHI_][q];
        const Tensor<1,Nsd,Number>&  grad_u   = fields.grad[TEMP_][q];
        const Tensor<1,Nsd,Number>&  grad_phi = fields.grad[PHI_][q];
        const double                 u_n      = fields.val_old[TEMP_][q];
        const double                 phi_n    = fields.val_old[PHI_][q];

        //nonlinear constitutive quantities
        Number lambda, tau;
        Tensor<1,Nsd,Number> dlambda_dq;
        model_.interface_coefficients(grad_phi, lambda, dlambda_dq, tau);

        const Number dfdphi   = model_.bulk_driving_force(phi, u);
        const Number hprime   = model_.latent_heat_slope(phi);

        //time derivatives (backward Euler)
        const Number dudt     = (u - u_n)/dt_;
        const Number dphidt   = (phi - phi_n)/dt_;

        //gradient + anisotropy flux: lambda^2 grad phi + |grad phi|^2 lambda
    dlambda/dq
        const Number grad_phi2 = grad_phi*grad_phi;
        const Number lambda2   = lambda*lambda;
        const Number coef      = grad_phi2*lambda;
        const Tensor<1,Nsd,Number> phi_flux = lambda2*grad_phi + coef*dlambda_dq;
```

```
36
37          const double JxW = fe_values.JxW(q);
38
39          for(const unsigned int i : fe_values.dof_indices()){
40              //shape functions of each field (zero if dof i belongs to another
    field)
41              const double          N_u       = fe_values[field_ext[TEMP_]].value(i,
    q);
42              const Tensor<1,Nsd>   gradN_u   = fe_values[field_ext[TEMP_]].gradient
    (i,q);
43              const double          N_phi     = fe_values[field_ext[PHI_]].value(i,q
    );
44              const Tensor<1,Nsd>   gradN_phi = fe_values[field_ext[PHI_]].gradient(
    i,q);
45
46              //Eq. 6a : temperature
47              const Number R_u   = N_u*(dudt - 0.5*hprime*dphidt) + D*(gradN_u*
    grad_u);
48              //Eq. 6b : phase field
49              const Number R_phi = N_phi*(tau*dphidt + dfdphi) + gradN_phi*phi_flux;
50
51              Rlocal[i] += (R_u + R_phi)*JxW;
52          }
53      }
54  }
```

ConstitutiveModel.hpp

```
1   #pragma once
2
3   class ConstitutiveModel{
4       public:
5           ConstitutiveModel(const double D, const double eps4, const double lambda0,
6                             const double tau0, const double xi)
7           : D_(D), eps4_(eps4), lambda0_(lambda0), tau0_(tau0), xi_(xi) {}
8
9           double diffusivity() const { return D_; }
10
11          //bulk driving force  df/dphi = g'(phi) + xi u p'(phi) = -phi + phi^3 + xi
     u (1 - phi^2)^2
12          template <typename T>
13          T bulk_driving_force(const T& phi, const T& u) const{
14              const T one_m_phi2 = 1.0 - phi*phi;
15              return -phi + phi*phi*phi + xi_*u*one_m_phi2*one_m_phi2;
16          }
17
18          //latent heat function h(phi) and its slope h'(phi) (U = u - h(phi)/2);
    here h = phi
19          template <typename T>
20          T latent_heat_slope(const T& /*phi*/) const{ return T(1.0); }
21
22          //four-fold anisotropy a_s(q) = (1-3eps4) + 4 eps4 (qx^4+qy^4)/|q|^4, q =
    grad phi
23          //returns a_s and d a_s / d q
24          template <int dim, typename T>
25          void anisotropy(const Tensor<1,dim,T>& q, T& a, Tensor<1,dim,T>& da_dq)
    const{
26              static_assert(dim == 2, "anisotropy() is written for 2D");
```

```
            const double delta = 1e-12; //regularisation where grad phi = 0 (bulk)
            const T qx = q[0], qy = q[1];
            const T q2 = qx*qx + qy*qy + delta;
            const T q4 = q2*q2;
            const T q6 = q4*q2;
            const T S  = (qx*qx*qx*qx + qy*qy*qy*qy)/q4;
            a = (1.0 - 3.0*eps4_) + 4.0*eps4_*S;
            da_dq[0] = 4.0*eps4_*(4.0*qx*qy*qy*(qx*qx - qy*qy)/q6);
            da_dq[1] = 4.0*eps4_*(4.0*qx*qx*qy*(qy*qy - qx*qx)/q6);
        }

        //interface coefficients from the anisotropy:
        //lambda(n) = lambda0 a_s,  d lambda / d q,  tau(n) = tau0 a_s^2
        template <int dim, typename T>
        void interface_coefficients(const Tensor<1,dim,T>& q, T& lambda, Tensor<1,dim,T>& dlambda_dq, T& tau) const{
            T a;
            Tensor<1,dim,T> da_dq;
            anisotropy(q, a, da_dq);
            lambda     = lambda0_*a;
            dlambda_dq = lambda0_*da_dq;
            tau        = tau0_*a*a;
        }

    private:
        const double D_, eps4_, lambda0_, tau0_, xi_;
};
```

OutputWriter.hpp

```
#pragma once

template <unsigned int Nsd, unsigned int BfOrder>
class OutputWriter{
    public:
        explicit OutputWriter(const std::string& output_dir);

        //field_names: one name per component of the FESystem, e.g. {"u", "phi"}
        void write_vtu_and_pvd(const DoFHandler<Nsd>& dof_handler, const Vector<double>& solution,
                               const std::vector<std::string>& field_names,
                               const unsigned int timestep, std::ofstream & pvd_output);

        void initiate_pvd(std::ofstream &pvd_output);
        void finish_pvd(std::ofstream &pvd_output);

        const std::string& get_output_dir() const { return output_dir_; }//return output directory location

    private:
        std::string output_dir_;
};

#include "OutputWriter.tpp"
```

OutputWriter.tpp

```cpp
#pragma once

template <unsigned int Nsd, unsigned int BfOrder>
OutputWriter<Nsd,BfOrder>::OutputWriter(const std::string& output_dir)
: output_dir_(output_dir)
{
    std::filesystem::create_directories(output_dir_);
}

template <unsigned int Nsd, unsigned int BfOrder>
void OutputWriter<Nsd,BfOrder>::write_vtu_and_pvd(const DoFHandler<Nsd>&
    dof_handler, const Vector<double>& solution,
                                                  const std::vector<std::string>&
    field_names,
                                                  const unsigned int timestep, std
    ::ofstream &pvd_output){
    DataOut<Nsd> data_out;
    data_out.attach_dof_handler(dof_handler);
    data_out.add_data_vector(solution, field_names);

    data_out.build_patches();

    std::string filename = "solution-" + dealii::Utilities::int_to_string(timestep
    , 7) + ".vtu"; //7 is for padding integer names in the file names
    std::ofstream vtu_output(output_dir_ + "/" + filename);
    data_out.write_vtu(vtu_output);
    pvd_output << "    <DataSet timestep=\"" << timestep << "\" group=\"\" part
    =\"0\" file=\"" << filename << "\"/>\n"; //append to pvd file (same folder)
}

template <unsigned int Nsd, unsigned int BfOrder>
void OutputWriter<Nsd,BfOrder>::initiate_pvd(std::ofstream &pvd_output){
    pvd_output << "<?xml version=\"1.0\"?>\n"
               << "<VTKFile type=\"Collection\" version=\"0.1\" "
               << "byte_order=\"LittleEndian\" "
               << "compressor=\"vtkZLibDataCompressor\">\n"
               << "  <Collection>\n";
}

template <unsigned int Nsd, unsigned int BfOrder>
void OutputWriter<Nsd,BfOrder>::finish_pvd(std::ofstream &pvd_output){
    pvd_output << "  </Collection>\n" << "</VTKFile>\n";
    pvd_output.close();
}
```